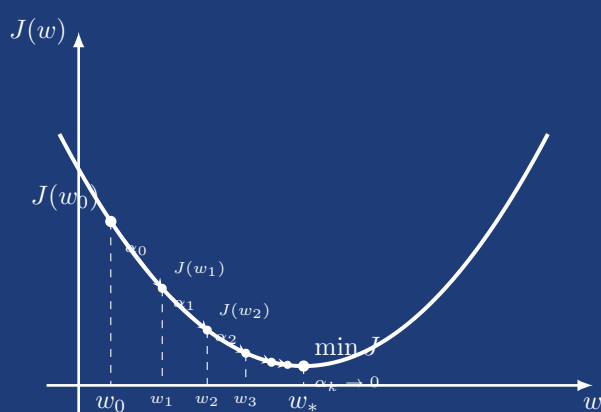


UNIVERSITÀ DEGLI STUDI LA SAPIENZA, ROMA

Alessandro Lo Curcio • 16 agosto 2026



Selezione Dinamica della Dimensione del Campione

in Metodi di Ottimizzazione per il Machine Learning

*Algoritmi adattivi a campione dinamico e
sottocampionamento dell'Hessiana
per problemi su larga scala*

Indice

1	Formulazione del Problema	2
1.1	Apprendimento supervisionato su larga scala	2
1.2	La sfida computazionale	2
1.3	Il trade-off	3
1.4	Formulazione del problema	3
1.5	Ipotesi su J	3
2	Algoritmi Proposti	5
2.1	Metodo del Gradiente a Campione Dinamico	5
2.1.1	Formulazione della dinamica del batch	5
2.1.2	Formulazione matematica della condizione di accettazione	5
2.1.3	Regola di aggiornamento del batch	8
2.1.4	Pseudocodice (Algoritmo)	8
2.1.5	Convergenza deterministica	9
2.1.6	Analisi Stocastica e Complessità del Metodo	13
2.2	Metodo di Newton-CG con Campionamento Dinamico	21
2.2.1	Struttura del metodo	21
2.2.2	Criterio di terminazione del gradiente coniugato	22
2.2.3	Pseudocodice (Algoritmo Newton-CG)	25
2.3	Metodo Newton-CG per Modelli con Regularizzazione L_1	28
2.3.1	Formulazione del Problema	28
2.3.2	Il gradiente generalizzato	28
2.3.3	Identificazione dell'Active Set e della Faccia Ortante	30
2.3.4	Fase di Minimizzazione nel Sottospazio	31
2.3.5	Ricerca Lineare Proiettata	31
2.3.6	Algoritmo Completo	32
3	Visualizzazione Interattiva	32
3.1	Metodo del Gradiente a Campione Dinamico	34
3.2	Metodo di Newton-CG con Campionamento Dinamico	37
		37
3.3	Metodo Newton-CG per Modelli con Regularizzazione L_1	42
4	Appendice	47
4.1	Dimostrazione delle disuguaglianze di Polyak–Łojasiewicz	47
4.2	Dimostrazione del fattore di correzione per popolazione finita	50
4.3	Dimostrazione delle formule per α_k e β_k nel Metodo del Gradiente Coniugato	52
4.4	Spiegazione delle condizioni sul passo α_k	55

1 Formulazione del Problema

1.1 Apprendimento supervisionato su larga scala

Nell'apprendimento supervisionato, si dispone di un insieme di esempi etichettati $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, dove:

- $x_i \in \mathbb{R}^m$ è il vettore delle *feature* (caratteristiche) dell' i -esimo esempio,
- $y_i \in \mathcal{Y}$ è la corrispondente *etichetta* (valore da predire),
- $N \in \mathbb{N}$ è il numero totale di esempi, potenzialmente dell'ordine di 10^6 – 10^9 .

Si assume che le coppie (x_i, y_i) siano realizzazioni i.i.d. di una variabile casuale con distribuzione di probabilità congiunta P su $\mathcal{X} \times \mathcal{Y}$. L'obiettivo ideale è minimizzare il rischio ovvero l'attesa della loss sullo spazio $\mathcal{X} \times \mathcal{Y}$:

$$\min_{w \in \mathbb{R}^m} R(w) = \mathbb{E}_{(x,y) \sim P} \ell(f(w, x), y) = \int_{\mathcal{X} \times \mathcal{Y}} \ell(f(w; x), y) dP(x, y)$$

dove:

- $f(w; x) = w^T x$ è il *predittore lineare* parametrizzato da w ,
- $\ell : \mathbb{R} \times \mathcal{Y} \rightarrow \mathbb{R}_{\geq 0}$ è la *funzione di perdita* convessa.

Poiché P è ignota, si minimizza invece il **rischio empirico**:

Definizione

Funzione Obiettivo Empirica.

$$\min_{w \in \mathbb{R}^m} J(w) = \min_{w \in \mathbb{R}^m} \frac{1}{N} \sum_{i=1}^N \ell(w; i), \quad (1)$$

dove abbiamo usato la notazione abbreviata $\ell(w; i) \triangleq \ell(f(w; x_i), y_i)$

- $w \in \mathbb{R}^m$: parametri del modello (variabile di ottimizzazione),
- N : dimensione del dataset,
- $\ell(w; i)$: perdita del modello w sull'esempio i -esimo.

1.2 La sfida computazionale

Il gradiente esatto di J richiede la somma su tutti gli N esempi:

$$\nabla J(w) = \frac{1}{N} \sum_{i=1}^N \nabla \ell(w; i).$$

Il costo di ogni $\nabla \ell(w; i)$ è $O(m)$, siccome deve essere valutato N volte, ogni valutazione del gradiente di J in un punto w costa $O(Nm)$ operazioni. Per $N = 10^8$ e $m = 10^4$ si tratta di 10^{12} operazioni *per singola iterazione* ovvero un costo che non è praticabile.

1.3 Il trade-off

Il problema è che per minimizzare J dobbiamo calcolare $\nabla J(w)$ in tanti punti w , una volta per ogni iterazione, finché non troviamo il minimo; in particolare un singolo gradiente di J costa $O(Nm)$. Se N è enorme, una iterazione è già costosissima, e di iterazioni ne servono tante. Le strade possibili sono le seguenti: il metodo stocastico (SGD) usa un solo esempio per iterazione: il costo è basso, ma il gradiente è rumoroso, quindi servono tante iterazioni e la convergenza è lenta. Il batch completo usa tutti i dati: il gradiente è esatto, quindi servono poche iterazioni, ma ogni iterazione costa $O(Nm)$ ed è impraticabile. Il mini-batch fisso è un compromesso: usa un numero intermedio di esempi, bilanciando costo e accuratezza, ma la dimensione del batch è arbitraria e fissa.

La soluzione proposta è il metodo del gradiente a campione dinamico: si parte con un batch piccolo per fare progressi veloci, e lo si aumenta gradualmente quando serve precisione, in questo modo il costo computazionale si concentra solo dove serve.

1.4 Formulazione del problema

Definizione

Perdita media sul campione $\mathcal{S}_k$: ad ogni iterazione k si seleziona un sottoinsieme $\mathcal{S}_k \subseteq \{1, \dots, N\}$ di dimensione $n_k = |\mathcal{S}_k|$ e si definisce la funzione

$$J_{\mathcal{S}_k}(w) = \frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \ell(w; i).$$

- $\mathcal{S}_k$: campione casuale di indici all'iterazione k ,
- $n_k = |\mathcal{S}_k|$: dimensione del batch, *variabile* con k ,
- $J_{\mathcal{S}_k}(w)$: approssimazione stocastica dell'obiettivo reale $J(w)$.

L'algoritmo definisce una regola per aggiornare w_k e per scegliere n_{k+1} in funzione delle informazioni disponibili all'iterazione k .

1.5 Ipotesi su J

Per garantire le proprietà teoriche degli algoritmi, si assumono le seguenti condizioni.

Definizione

Convessità forte e lisciezza: la funzione $J : \mathbb{R}^m \rightarrow \mathbb{R}$ è *uniformemente convessa* e ha gradiente *Lipschitz continuo*: esistono costanti $0 < \lambda \leq L$ tali che, per ogni $w, d \in \mathbb{R}^m$,

$$\lambda \|d\|_2^2 \leq d^T \nabla^2 J(w) d \leq L \|d\|_2^2.$$

- $\lambda > 0$: costante di convessità forte. Impone che $\nabla^2 J(w) \succeq \lambda I$ per ogni w , il che garantisce che J abbia un unico minimizzatore w^* .
- L : costante di Lipschitz del gradiente. Impone che $\|\nabla J(x) - \nabla J(y)\|_2 \leq L\|x - y\|_2$ per ogni x, y , il che equivale a $\nabla^2 J(w) \preceq LI$ per ogni w .

- $\kappa = L/\lambda$: numero di condizionamento (stima superiore). Per una data Hessiana $\nabla^2 J(w)$ (matrice simmetrica), il numero di condizionamento in norma 2 sarebbe $\lambda_{\max}(\nabla^2 J(w))/\lambda_{\min}(\nabla^2 J(w))$. Dato che gli autovallori sono ovunque compresi tra λ e L , si ha $\kappa \leq L/\lambda$, $\kappa = L/\lambda$ viene usato come stima superiore uniforme. Se $\kappa \gg 1$, il problema è mal condizionato e più difficile da ottimizzare.

Nota

In pratica, la convessità forte non è sempre verificata (ad esempio, la perdita logistica non è fortemente convessa). Tuttavia, è sufficiente aggiungere un termine di regolarizzazione $\frac{\gamma}{2} \|w\|^2$ alla funzione obiettivo J se è convessa, per indurre convessità forte con $\lambda = \gamma$.

Nota

Disuguaglianze fondamentali (si veda l'Appendice 4 per la dimostrazione completa). Sia w_* l'unico minimo di J e, per semplificare le notazioni, poniamo $J(w_*) = 0$. Allora con le ipotesi strutturali esplicitate sopra valgono le seguenti disuguaglianze:

$$\|\nabla J(w)\|_2^2 \geq 2\lambda J(w), \quad J(w) \geq \frac{\lambda}{2L^2} \|\nabla J(w)\|_2^2.$$

2 Algoritmi Proposti

2.1 Metodo del Gradiente a Campione Dinamico

2.1.1 Formulazione della dinamica del batch

Il metodo del gradiente (o *steepest descent*) è l'algoritmo di ottimizzazione più semplice: a ogni passo, si scende nella direzione opposta al gradiente. La variante proposta non usa il gradiente esatto $\nabla J(w_k)$ (troppo costoso), ma un'approssimazione basata su un campione:

$$g_k = \nabla J_{\mathcal{S}_k}(w_k) = \frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \nabla \ell(w_k; i).$$

Vogliamo sapere quando il campione $\mathcal{S}_k$ è abbastanza grande da garantire che $-g_k$ sia davvero una direzione di discesa per J .

La direzione $-g_k$ è di discesa per J se l'errore del gradiente $\|g_k - \nabla J(w_k)\|$ è piccolo rispetto a $\|g_k\|$ (dimostrazione sotto). Questo errore non è calcolabile esattamente, ma può essere *stimato* dalla varianza campionaria di $\{\nabla \ell(w_k; i)\}_{i \in \mathcal{S}_k}$: se i gradienti dei singoli esempi nel batch sono coerenti tra loro (bassa varianza), allora la loro media è un buon estimatore di $\nabla J(w_k)$.

Nel metodo del gradiente classico, si richiede che la direzione di discesa d_k soddisfi $\nabla J(w_k)^T d_k < 0$. Qui, poiché g_k è la direzione di discesa, che approssima $\nabla J(w_k)$, abbiamo $\nabla J(w_k)^T (-g_k) = -\nabla J(w_k)^T g_k$. Se g_k è sufficientemente vicino a $\nabla J(w_k)$ in norma relativa, allora $-g_k$ è effettivamente una direzione di discesa.

2.1.2 Formulazione matematica della condizione di accettazione

La direzione $d_k = -g_k$ è di discesa per J se l'errore del gradiente stimato rispetto a quello reale è sufficientemente piccolo in norma relativa:

$$\|g_k - \nabla J(w_k)\|_2 \leq \theta \|g_k\|_2, \quad \theta \in [0, 1).$$

Per capire perché questo garantisce la discesa, poniamo $e_k = g_k - \nabla J(w_k)$, cioè $g_k = \nabla J(w_k) + e_k$. Allora

$$\nabla J(w_k)^T g_k = \nabla J(w_k)^T (\nabla J(w_k) + e_k) = \|\nabla J(w_k)\|^2 + \nabla J(w_k)^T e_k.$$

Poiché $\nabla J(w_k) = g_k - e_k$, si ha anche

$$\nabla J(w_k)^T g_k = (g_k - e_k)^T g_k = \|g_k\|^2 - e_k^T g_k.$$

Per ogni numero reale x vale $x \geq -|x|$; quindi $e_k^T g_k \geq -|e_k^T g_k|$. Usando Cauchy-Schwarz, $|e_k^T g_k| \leq \|e_k\| \|g_k\|$, da cui $e_k^T g_k \geq -\|e_k\| \|g_k\|$. Sostituendo,

$$\nabla J(w_k)^T g_k \geq \|g_k\|^2 - \|e_k\| \|g_k\| = \|g_k\| (\|g_k\| - \|e_k\|).$$

Affinché $\nabla J(w_k)^T g_k > 0$ (cioè $-g_k$ sia di discesa) è sufficiente che $\|g_k\| - \|e_k\| > 0$, ovvero $\|e_k\| < \|g_k\|$. La condizione $\|e_k\| \leq \theta \|g_k\|$ con $\theta < 1$ implica $\|e_k\| < \|g_k\|$, dunque garantisce la discesa.

Poiché il gradiente della popolazione totale $\nabla J(w_k)$ è ignoto, non è possibile valutare questa condizione in modo deterministico a ogni iterazione. È importante distinguere due livelli: la condizione puntuale $\|g_k - \nabla J(w_k)\| \leq \theta \|g_k\|$ è quella usata nel Teorema deterministico per garantire la discesa a ogni passo; la Condizione di Controllo della Varianza (CCV) che introdurremo tra poco è invece una condizione *in aspettativa*, sufficiente per l'analisi di convergenza in media ma non una garanzia puntuale. Il mini-batch $\mathcal{S}_k$ viene estratto casualmente dal dataset, rendendo g_k una variabile aleatoria con $\mathbb{E}_{\mathcal{S}_k}[g_k] = \nabla J(w_k)$. Passare dal controllo dell'errore puntuale $\|g_k - \nabla J(w_k)\|$ al controllo del suo valore atteso permette di aggirare l'ignoranza su $\nabla J(w_k)$:

$$\begin{aligned}\mathbb{E}_{\mathcal{S}_k} [\|g_k - \mathbb{E}_{\mathcal{S}_k}[g_k]\|_2^2] &= \mathbb{E}_{\mathcal{S}_k} \left[\sum_{j=1}^m \left(g_k^{(j)} - \mathbb{E}_{\mathcal{S}_k}[g_k^{(j)}] \right)^2 \right] \\ &= \sum_{j=1}^m \mathbb{E}_{\mathcal{S}_k} \left[\left(g_k^{(j)} - \mathbb{E}_{\mathcal{S}_k}[g_k^{(j)}] \right)^2 \right] \\ &= \sum_{j=1}^m \text{Var}_{\mathcal{S}_k} \left(g_k^{(j)} \right) = \|\text{Var}_{\mathcal{S}_k}(g_k)\|_1.\end{aligned}$$

Decomposizione della varianza dello stimatore. Definiamo la varianza della popolazione (vettore):

$$\begin{aligned}\mathcal{V} &:= \mathbb{E}_I [(\nabla \ell(w_k; I) - \mathbb{E}_I[\nabla \ell(w_k; I)])^2] \\ &= \text{Var}_I(\nabla \ell(w_k; I)) = \frac{1}{N} \sum_{i=1}^N (\nabla \ell(w_k; i) - \nabla J(w_k))^2 \in \mathbb{R}^m,\end{aligned}$$

dove il quadrato è inteso componente per componente, e distinguiamo I variabile aleatoria distribuita uniformemente in $\{1, \dots, N\}$ con i realizzazione di tale variabile.

Caso con reinserimento (indipendenza):

$$\begin{aligned}\text{Var}(g_k) &= \text{Var} \left(\frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \nabla \ell(w_k; i) \right) = \frac{1}{n_k^2} \sum_{i \in \mathcal{S}_k} \text{Var}(\nabla \ell(w_k; i)) \\ &= \frac{1}{n_k^2} \underbrace{\text{Var}(\nabla \ell(w_k; i)) + \dots + \text{Var}(\nabla \ell(w_k; i))}_{n_k \text{ volte}} = \frac{\mathcal{V}}{n_k}.\end{aligned}$$

Anche qui la divisione va fatta componente per componente, siccome $\mathcal{V}$ è un vettore.

Caso senza reinserimento (dipendenza):

$$\text{Var}(\nabla J_{\mathcal{S}_k}(w_k)) = \frac{\mathcal{V}}{n_k} \cdot \frac{N - n_k}{N - 1}.$$

La dimostrazione del fattore che si aggiunge qui è riportata nell'appendice. Per $N \gg n_k$ (condizione tipica nel Machine Learning), il fattore di correzione $\frac{N - n_k}{N - 1}$ è circa 1, quindi $\text{Var}(g_k) \approx \mathcal{V}/n_k$: la condizione esatta si semplifica nella forma pratica riportata sotto.

Poiché $\mathcal{V}$ è ignoto, lo sostituiamo con la sua stima campionaria sul batch corrente:

$$\widehat{\mathcal{V}} := \text{Var}_{i \in S_k}(\nabla \ell(w_k; i)) = \frac{1}{n_k - 1} \sum_{i \in S_k} (\nabla \ell(w_k; i) - \nabla J_{S_k}(w_k))^2.$$

Otteniamo così la **Condizione di Controllo della Varianza (CCV)**:

Teorema

$$\frac{\|\widehat{\mathcal{V}}\|_1}{n_k} \leq \theta^2 \|\nabla J_{S_k}(w_k)\|_2^2,$$

dove:

- $\|\cdot\|_1$ somma le varianze componente per componente,
- $\theta \in (0, 1)$ è la tolleranza impostata dall'utente,
- n_k è la dimensione del batch corrente.

Questa è la forma pratica per $N \gg n_k$ (corrispondente al limite $N \rightarrow \infty$ della condizione esatta che include il fattore $\frac{N-n_k}{N-1}$ derivata sopra). La condizione verifica che la varianza stimata dello stimatore sia sufficientemente piccola rispetto al gradiente corrente. Se non è soddisfatta, si aumenta n_k .

Nota

Dalla condizione di discesa (1) sappiamo che, per garantire che $-g_k$ sia una direzione di discesa, dobbiamo avere:

$$\|e_k\|_2 \leq \theta \|g_k\|_2, \quad e_k := g_k - \nabla J(w_k).$$

Elevando al quadrato entrambi i membri:

$$\|e_k\|_2^2 \leq \theta^2 \|g_k\|_2^2.$$

Poiché $\nabla J(w_k)$ è ignoto, non possiamo calcolare $\|e_k\|_2^2$ esattamente. Tuttavia, poiché $\mathbb{E}[g_k] = \nabla J(w_k)$, si ha:

$$\mathbb{E}[\|e_k\|_2^2] = \mathbb{E}[\|g_k - \mathbb{E}[g_k]\|_2^2] = \|\text{Var}(g_k)\|_1.$$

Stimando $\|\text{Var}(g_k)\|_1 \approx \frac{\|\widehat{\mathcal{V}}\|_1}{n_k}$, la disuguaglianza in valore atteso diventa esattamente la CCV:

$$\frac{\|\widehat{\mathcal{V}}\|_1}{n_k} \leq \theta^2 \|g_k\|_2^2.$$

Il parametro θ controlla la tolleranza sull'errore relativo $\|e_k\|/\|g_k\|$; elevandolo al quadrato si preserva la coerenza dimensionale con la varianza (che è un errore quadratico medio).

2.1.3 Regola di aggiornamento del batch

Se la condizione CCV non è soddisfatta, si stima la nuova dimensione minima del batch. Supponendo che, per aumenti gradualmente, la varianza campionaria e la norma del gradiente non cambino significativamente (ovvero la stima della varianza per predire la dimensione del batch k viene fatta con la varianza del batch $k-1$) si ottiene:

Definizione

Regola di Aggiornamento del Batch.

$$n_k^{\text{new}} = \left\lceil \frac{\|\hat{\mathcal{V}}\|_1}{\theta^2 \|\nabla J_{\mathcal{S}_k}(w_k)\|_2^2} \right\rceil_1$$

dove $\hat{\mathcal{V}} := \text{Var}_{i \in \mathcal{S}_k}(\nabla \ell(w_k; i))$ è la varianza campionaria.

Nota

Quando siamo lontani dal minimo, $\|\nabla J_{\mathcal{S}_k}(w_k)\|_2^2$ è grande: la formula di aggiornamento del batch dà un n piccolo, quindi il batch resta piccolo. Avvicinandoci al minimo, il gradiente si azzerizza e il denominatore diventa piccolo: la formula impone un batch grande. Il batch cresce automaticamente dove serve precisione.

2.1.4 Pseudocodice (Algoritmo)

Algoritmo

Gradiente a Campione Dinamico

Input: w_0 (punto iniziale), $\mathcal{S}_0$ (batch iniziale), $\theta \in (0, 1)$ (tolleranza).

Ripeti per ogni $k \in \mathbb{N}$ fino a convergenza:

1. $g_k = \nabla J_{\mathcal{S}_k}(w_k)$.
2. $\hat{\mathcal{V}} = \text{Var}_{i \in \mathcal{S}_k}(\nabla \ell(w_k; i)) = \frac{1}{n_k - 1} \sum_{i \in \mathcal{S}_k} (\nabla \ell_i(w_k) - g_k)^2$.
3. Se $\frac{\|\hat{\mathcal{V}}\|_1}{n_k} > \theta^2 \|g_k\|_2^2$ aumenta il batch: $n_k \leftarrow \left\lceil \frac{\|\hat{\mathcal{V}}\|_1}{\theta^2 \|g_k\|_2^2} \right\rceil$.
4. Esegui una line search: trova $\alpha_k > 0$ tale che $J_{\mathcal{S}_k}(w_k - \alpha_k g_k) < J_{\mathcal{S}_k}(w_k)$.
5. Aggiorna: $w_{k+1} = w_k - \alpha_k g_k$.
6. Seleziona un nuovo campione $\mathcal{S}_{k+1}$ di dimensione n_k (la nuova dimensione, eventualmente aumentata).

¹ Nell'implementazione si aggiunge 1 per evitare che, quando il rapporto è esattamente intero, il batch non cresca, causando ripetute violazioni della CCV per fluttuazioni numeriche.

2.1.5 Convergenza deterministica

Consideriamo l'iterazione a passo fisso

$$w_{k+1} = w_k - \alpha g_k, \quad \alpha = \frac{1 - \theta}{L},$$

dove g_k è un'approssimazione di $\nabla J(w_k)$ che soddisfa la condizione di accuratezza

$$\|g_k - \nabla J(w_k)\|_2 \leq \theta \|g_k\|_2, \quad \theta \in [0, 1).$$

Lemma 1. *Sotto la condizione di accuratezza si hanno le seguenti disuguaglianze:*

$$\|\nabla J(w_k)\| \leq (1 + \theta)\|g_k\|, \quad \|\nabla J(w_k)\| \geq (1 - \theta)\|g_k\|.$$

Inoltre,

$$\nabla J(w_k)^T g_k \geq (1 - \theta)\|g_k\|^2.$$

Dimostrazione. Poniamo $e_k = g_k - \nabla J(w_k)$. Allora per definizione $\nabla J(w_k) = g_k - e_k$.

Prima dimostrazione: $\|\nabla J(w_k)\| \leq (1 + \theta)\|g_k\|$. Applicando la disuguaglianza triangolare $\|x + y\| \leq \|x\| + \|y\|$ con $x = g_k$ e $y = -e_k$:

$$\|\nabla J(w_k)\| = \|g_k + (-e_k)\| \leq \|g_k\| + \|-e_k\| = \|g_k\| + \|e_k\|.$$

Dalla condizione di accuratezza, $\|e_k\| \leq \theta\|g_k\|$. Sostituendo:

$$\|\nabla J(w_k)\| \leq \|g_k\| + \theta\|g_k\| = (1 + \theta)\|g_k\|.$$

Seconda dimostrazione: $\|\nabla J(w_k)\| \geq (1 - \theta)\|g_k\|$. Partiamo dalla disuguaglianza triangolare standard $\|a\| = \|a - b + b\| \leq \|a - b\| + \|b\|$, valida per ogni a, b . Ponendo $a = g_k$ e $b = e_k$:

$$\|g_k\| \leq \|g_k - e_k\| + \|e_k\| = \|\nabla J(w_k)\| + \|e_k\|.$$

Quindi $\|\nabla J(w_k)\| \geq \|g_k\| - \|e_k\|$. Sostituendo $\|e_k\| \leq \theta\|g_k\|$:

$$\|\nabla J(w_k)\| \geq \|g_k\| - \theta\|g_k\| = (1 - \theta)\|g_k\|.$$

Dimostrazione di $\nabla J(w_k)^T g_k \geq (1 - \theta)\|g_k\|^2$. Eleviamo al quadrato la condizione di accuratezza:

$$\|g_k - \nabla J(w_k)\|^2 \leq \theta^2 \|g_k\|^2.$$

Sviluppando il quadrato del binomio:

$$\|g_k\|^2 - 2\nabla J(w_k)^T g_k + \|\nabla J(w_k)\|^2 \leq \theta^2 \|g_k\|^2.$$

Isoliamo il termine con il prodotto scalare portando gli altri termini a destra:

$$-2\nabla J(w_k)^T g_k \leq \theta^2 \|g_k\|^2 - \|g_k\|^2 - \|\nabla J(w_k)\|^2 = (\theta^2 - 1)\|g_k\|^2 - \|\nabla J(w_k)\|^2.$$

Moltiplichiamo entrambi i membri per -1 :

$$2\nabla J(w_k)^T g_k \geq (1 - \theta^2)\|g_k\|^2 + \|\nabla J(w_k)\|^2.$$

Utilizziamo ora la disuguaglianza $\|\nabla J(w_k)\| \geq (1-\theta)\|g_k\|$ dimostrata in precedenza, elevandola al quadrato:

$$\|\nabla J(w_k)\|^2 \geq (1-\theta)^2\|g_k\|^2.$$

Sostituiamo questa stima nella disuguaglianza del Lemma:

$$2\nabla J(w_k)^T g_k \geq (1-\theta^2)\|g_k\|^2 + (1-\theta)^2\|g_k\|^2.$$

Raccogliamo $\|g_k\|^2$ e semplifichiamo i coefficienti:

$$2\nabla J(w_k)^T g_k \geq [(1-\theta^2)+(1-2\theta+\theta^2)]\|g_k\|^2 = [1-\theta^2+1-2\theta+\theta^2]\|g_k\|^2 = 2(1-\theta)\|g_k\|^2.$$

Dividendo entrambi i membri per 2 si ottiene la tesi:

$$\nabla J(w_k)^T g_k \geq (1-\theta)\|g_k\|^2.$$

□

Teorema

Teorema di Convergenza Deterministica. Sia $J : \mathbb{R}^m \rightarrow \mathbb{R}$ una funzione λ -fortemente convessa ($\nabla^2 J(w) \succeq \lambda I$ per ogni w) e con gradiente L -Lipschitz ($\nabla^2 J(w) \preceq LI$ per ogni w). Si assuma che a ogni iterazione valga la condizione di accuratezza e si scelga il passo $\alpha = \frac{1-\theta}{L}$. Allora

$$\underbrace{J(w_{k+1}) - J(w^*)}_{=:\varepsilon_{k+1}} \leq \left(1 - \frac{(1-\theta)\lambda}{L}\right) \underbrace{(J(w_k) - J(w^*))}_{=:\varepsilon_k}.$$

Di conseguenza:

- Il fattore di riduzione per iterazione è $\rho = 1 - \frac{(1-\theta)\lambda}{L} \in (0, 1)$;
- Il numero di iterazioni per ottenere $J(w_k) - J(w^*) \leq \varepsilon$ è al più

$$k \geq \frac{L}{(1-\theta)\lambda} \log\left(\frac{J(w_0) - J(w^*)}{\varepsilon}\right) = O\left(\frac{L}{\lambda} \log \frac{1}{\varepsilon}\right).$$

Nota

Confronto con il risultato di Nocedal (Theorem 4.1). Il fattore di contrazione qui ottenuto è $1 - \frac{(1-\theta)\lambda}{L}$, mentre Nocedal dimostra $1 - \frac{(1-\theta)\lambda}{2L}$ (eq. 4.9 del paper). Ora $1 - \frac{(1-\theta)\lambda}{L} < 1 - \frac{(1-\theta)\lambda}{2L}$, cioè abbiamo un fattore di contrazione più piccolo (convergenza più rapida). Esso è dovuto all'uso della disuguaglianza di Polyak–Lojasiewicz nella forma forte $\|\nabla J(w)\|^2 \geq 2\lambda J(w)$ (dimostrata nell'appendice 4), invece della forma più debole $\|\nabla J(w)\|^2 \geq \lambda(J(w) - J(w_*))$ (eq. 4.4–4.5) utilizzata da Nocedal. La forma più forte è resa possibile dall'ipotesi di convessità forte ($\nabla^2 J \succeq \lambda I$), che implica direttamente $\|\nabla J(w)\|^2 \geq 2\lambda J(w)$ quando $J(w_*) = 0$.

Dimostrazione. Per semplificare poniamo $J(w^*) = 0$ (altrimenti si definisce $\tilde{J}(w) = J(w) - J(w^*)$, che soddisfa le stesse ipotesi di convessità e ha minimo nullo).

Per la L -lipschitzianità del gradiente, vale la seguente maggiorazione (conseguenza del teorema di Taylor con resto di Lagrange):

$$J(w_{k+1}) \leq J(w_k) + \nabla J(w_k)^T (w_{k+1} - w_k) + \frac{L}{2} \|w_{k+1} - w_k\|^2.$$

Sostituendo $\alpha = \frac{1-\theta}{L}$ e $w_{k+1} = w_k - \alpha g_k \Rightarrow w_{k+1} - w_k = -\alpha g_k$:

$$J(w_{k+1}) \leq J(w_k) - \frac{1-\theta}{L} \nabla J(w_k)^T g_k + \frac{L}{2} \left(\frac{1-\theta}{L}\right)^2 \|g_k\|^2.$$

Semplifichiamo il coefficiente del termine quadratico:

$$\frac{L}{2} \left(\frac{1-\theta}{L}\right)^2 = \frac{L}{2} \cdot \frac{(1-\theta)^2}{L^2} = \frac{(1-\theta)^2}{2L}.$$

Quindi:

$$J(w_{k+1}) \leq J(w_k) - \underbrace{\frac{1-\theta}{L} \nabla J(w_k)^T g_k + \frac{(1-\theta)^2}{2L} \|g_k\|^2}_{:=S}.$$

Moltiplichiamo la disuguaglianza del Lemma per $\frac{1-\theta}{2L}$:

$$\frac{1-\theta}{2L} \cdot 2 \nabla J(w_k)^T g_k \geq \frac{1-\theta}{2L} \left[(1-\theta^2) \|g_k\|^2 + \|\nabla J(w_k)\|^2 \right].$$

Semplificando il membro sinistro:

$$\frac{1-\theta}{L} \nabla J(w_k)^T g_k \geq \frac{(1-\theta)(1-\theta^2)}{2L} \|g_k\|^2 + \frac{1-\theta}{2L} \|\nabla J(w_k)\|^2.$$

Riorganizziamo la disuguaglianza appena scritta per isolare $-\frac{1-\theta}{L} \nabla J(w_k)^T g_k$, che compare nella definizione di S :

$$-\frac{1-\theta}{L} \nabla J(w_k)^T g_k \leq -\frac{(1-\theta)(1-\theta^2)}{2L} \|g_k\|^2 - \frac{1-\theta}{2L} \|\nabla J(w_k)\|^2.$$

Inseriamo questa disuguaglianza nella definizione di S :

$$J(w_{k+1}) \leq J(w_k) - \frac{(1-\theta)(1-\theta^2)}{2L} \|g_k\|^2 - \frac{1-\theta}{2L} \|\nabla J(w_k)\|^2 + \frac{(1-\theta)^2}{2L} \|g_k\|^2.$$

Raccogliamo i coefficienti di $\|g_k\|^2$:

$$-\frac{(1-\theta)(1-\theta^2)}{2L} + \frac{(1-\theta)^2}{2L} = -\frac{1-\theta}{2L} \left[(1-\theta^2) - (1-\theta) \right].$$

Calcoliamo la differenza tra parentesi:

$$(1-\theta^2) - (1-\theta) = 1-\theta^2 - 1 + \theta = \theta - \theta^2 = \theta(1-\theta).$$

Quindi:

$$-\frac{1-\theta}{2L} \cdot \theta(1-\theta) = -\frac{\theta(1-\theta)^2}{2L}.$$

La disuguaglianza diventa:

$$J(w_{k+1}) \leq J(w_k) - \frac{\theta(1-\theta)^2}{2L} \|g_k\|^2 - \frac{1-\theta}{2L} \|\nabla J(w_k)\|^2.$$

Poiché $\frac{\theta(1-\theta)^2}{2L} \|g_k\|^2 \geq 0$, il termine $-\frac{\theta(1-\theta)^2}{2L} \|g_k\|^2$ è minore o uguale a zero, trascurarlo può solo aumentare il membro destro e la disuguaglianza rimane valida:

$$J(w_{k+1}) \leq J(w_k) - \frac{1-\theta}{2L} \|\nabla J(w_k)\|^2.$$

Infine, dalla disuguaglianza di convessità forte dimostrata nell'Appendice 4, $\|\nabla J(w_k)\|^2 \geq 2\lambda J(w_k)$ (valida con $J(w_*) = 0$), si ottiene:

$$S \leq -\frac{1-\theta}{2L} \cdot 2\lambda J(w_k) = -\frac{\lambda(1-\theta)}{L} J(w_k).$$

Sostituendo nella disuguaglianza che definisce S :

$$J(w_{k+1}) \leq J(w_k) - \frac{\lambda(1-\theta)}{L} J(w_k) = \left(1 - \frac{\lambda(1-\theta)}{L}\right) J(w_k).$$

Applicando la disuguaglianza appena dimostrata ricorsivamente:

$$J(w_k) \leq \rho^k J(w_0), \quad \text{dove } \rho = 1 - \frac{\lambda(1-\theta)}{L}.$$

Poiché $\theta \in [0, 1)$, $\lambda > 0$ e $L > 0$, si ha $\rho \in (0, 1)$.

Imponiamo $J(w_k) \leq \varepsilon$:

$$\rho^k J(w_0) \leq \varepsilon \implies k \geq \frac{\log(J(w_0)/\varepsilon)}{\log(1/\rho)}.$$

Usando $\log(1/\rho) \geq 1 - \rho$:

$$1 - \rho = \frac{\lambda(1-\theta)}{L},$$

quindi:

$$k \geq \frac{L}{\lambda(1-\theta)} \log\left(\frac{J(w_0)}{\varepsilon}\right).$$

Questa è la stima del numero di iterazioni necessarie per ridurre l'errore al di sotto di ε . $\square$

2.1.6 Analisi Stocastica e Complessità del Metodo

Nell'analisi deterministica abbiamo assunto la condizione di accuratezza

$$\|g_k - \nabla J(w_k)\|_2 \leq \theta \|g_k\|_2, \quad \theta \in [0, 1), \quad (3.1)$$

garantendo che $-g_k$ sia una direzione di discesa per J e stabilendo la convergenza lineare (Teorema di Convergenza Deterministica).

Nel caso stocastico, $g_k = \nabla J_{\mathcal{S}_k}(w_k)$ è calcolato su un campione casuale $\mathcal{S}_k$ di dimensione n_k , rendendo g_k una variabile aleatoria. La condizione (3.1) non può essere garantita con probabilità 1 per ogni iterazione. Dobbiamo quindi sviluppare un'analisi in aspettativa, tenendo conto che n_k può variare nel corso delle iterazioni.

Impostazione del problema stocastico Definiamo

$$g_k := \nabla J_{\mathcal{S}_k}(w_k) = \frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \nabla \ell(w_k; i), \quad n_k := |\mathcal{S}_k|. \quad (3.2)$$

Per ogni w_k fissato, g_k è uno stimatore non distorto:

$$\mathbb{E}[g_k \mid w_k] = \nabla J(w_k). \quad (3.3)$$

Introduciamo l'ipotesi fondamentale di limitatezza della varianza dei gradienti individuali. Si noti che questa è un'ipotesi forte, necessaria per l'analisi di complessità teorica; in pratica può essere rilassata, ad esempio richiedendo la limitatezza solo in un intorno del minimo o utilizzando stime locali della varianza.

Nota

Ipotesi di limitatezza della varianza. $\exists \omega \in \mathbb{R}, \omega > 0$ tale che $\forall w_k$,

$$\|\text{Var}(\nabla \ell(w_k; I))\|_1 \leq \omega, \quad (3.4)$$

dove $\text{Var}(\nabla \ell(w_k; I)) := \mathbb{E}_I[(\nabla \ell(w_k; I) - \nabla J(w_k))^2]$ è inteso componente per componente.

Dal Lemma 2 abbiamo $\|\nabla J(w_k)\| \geq (1 - \theta)\|g_k\|$, da cui

$$\|g_k\|^2 \leq \frac{1}{(1 - \theta)^2} \|\nabla J(w_k)\|^2.$$

Dall'Appendice 4 (disuguaglianza $J(w) \geq \frac{\lambda}{2L^2} \|\nabla J(w)\|^2$) otteniamo $\|\nabla J(w_k)\|^2 \leq \frac{2L^2}{\lambda} J(w_k)$. Combinando con la convergenza lineare del Teorema di Convergenza Deterministica, $J(w_k) \leq (1 - \frac{(1-\theta)\lambda}{L})^k J(w_0)$, segue

$$\|g_k\|^2 \leq \frac{1}{(1 - \theta)^2} \frac{2L^2}{\lambda} J(w_k) \leq \frac{1}{(1 - \theta)^2} \frac{2L^2 J(w_0)}{\lambda} \left(1 - \frac{(1 - \theta)\lambda}{L}\right)^k. \quad (3.5)$$

Va osservato che questo bound ha natura circolare: la convergenza lineare del Teorema di Convergenza Deterministica, usata nella (3.5), richiede a sua volta che la CCV sia soddisfatta a ogni iterazione, ed è proprio questo che si intende garantire con la scelta di n_k . La (3.5) va quindi intesa come una stima guida/euristica per la scelta di n_k , non come una derivazione rigorosa, in analogia con quanto fa il paper originale. Questo bound adotta il tasso deterministico $\rho = 1 - \frac{(1-\theta)\lambda}{L}$ usato nel teorema precedente. Si noti che questo tasso è più favorevole (più rapido) rispetto a quello ottenuto da Byrd et al. (2012), che presenta un fattore 1/2 aggiuntivo a denominatore a causa di una diversa condizione di accuratezza. Dalla Condizione di Controllo della Varianza (CCV):

$$\frac{\overbrace{\|\text{Var}_{i \in \mathcal{S}_k}(\nabla \ell(w_k; i))\|_1}^{\hat{v}}}{n_k} \leq \theta^2 \|g_k\|^2. \quad (3.6)$$

Per garantire la (3.6) in media, dobbiamo scegliere n_k sufficientemente grande. Usando l'ipotesi (3.4), la condizione (3.6) è sicuramente soddisfatta se

$$\frac{\omega}{n_k} \leq \theta^2 \|g_k\|^2,$$

ovvero se

$$n_k \geq \frac{\omega}{\theta^2 \|g_k\|^2}. \quad (3.7)$$

Osserviamo che la (3.5) fornisce un *bound superiore* su $\|g_k\|^2$ che decresce geometricamente con tasso $\rho = 1 - (1 - \theta)\lambda/L$. Poiché nella (3.7) la dimensione del batch è inversamente proporzionale a $\|g_k\|^2$, al restringersi del gradiente n_k deve crescere per evitare che il rumore domini la discesa. Per stimare la velocità di crescita necessaria, usiamo il bound (3.5) per valutare il *tasso massimo* con cui il denominatore di (3.7)

può restringersi: sostituendo la (3.5) nella (3.7) si ottiene una condizione guida sulla crescita di n_k ,

$$n_k \geq \frac{\omega}{\theta^2 \cdot \frac{1}{(1-\theta)^2} \frac{2L^2 J(w_0)}{\lambda} \left(1 - \frac{(1-\theta)\lambda}{L}\right)^k} = \frac{\omega(1-\theta)^2 \lambda}{\theta^2 2L^2 J(w_0)} \left(1 - \frac{(1-\theta)\lambda}{L}\right)^{-k}. \quad (3.8)$$

Pertanto, per far decadere il rumore almeno alla stessa velocità con cui l'errore deterministico si riduce, n_k deve crescere geometricamente come

$$n_k \geq O\left(\left(1 - \frac{(1-\theta)\lambda}{L}\right)^{-k}\right).$$

Nell'analisi stocastica imponiamo quindi:

$$n_k = \lceil a^k \rceil, \quad a > 1. \quad (3.9)$$

Le ipotesi si dividono in due categorie complementari:

- **Ipotesi strutturali su J** ($\lambda I \preceq \nabla^2 J(w) \preceq LI$): garantiscono la convergenza lineare se la direzione di discesa è accurata (Teorema di Convergenza Deterministica), e forniscono la maggiorazione di Taylor (A.12), che sarà usata per derivare la disuguaglianza di convergenza in aspettativa.
- **Ipotesi stocastica** (3.4): garantisce che la varianza dello stimatore g_k decada come $\|\text{Var}(g_k)\|_1 \leq \omega/n_k$, rendendo il rumore controllabile aumentando il batch.

Iterazione stocastica a passo fisso Consideriamo l'iterazione

$$w_{k+1} = w_k - \frac{1}{L} g_k. \quad (3.10)$$

Nell'analisi stocastica si adotta il passo $1/L$, più lungo del passo deterministico $(1-\theta)/L$ poiché il fattore $(1-\theta)$ non è necessario quando il controllo dell'errore è affidato alla crescita del batch. Questo produce un tasso di contrazione deterministico $\alpha = 1 - \lambda/L$ più piccolo rispetto a $1 - (1-\theta)\lambda/L$, e quindi un reciproco $(1 - \lambda/L)^{-1}$ più grande: il vincolo su a ammette un bound superiore più alto.

Usando la (A.12) con $y = w_{k+1}$ e $w = w_k$:

$$J(w_{k+1}) \leq J(w_k) + \nabla J(w_k)^T (w_{k+1} - w_k) + \frac{L}{2} \|w_{k+1} - w_k\|^2. \quad (3.11)$$

Sostituendo $w_{k+1} - w_k = -\frac{1}{L} g_k$:

$$J(w_{k+1}) \leq J(w_k) - \frac{1}{L} \nabla J(w_k)^T g_k + \frac{1}{2L} \|g_k\|^2. \quad (3.12)$$

Passando al valore atteso condizionato a w_k :

$$\mathbb{E}[J(w_{k+1}) \mid w_k] \leq J(w_k) - \frac{1}{L} \nabla J(w_k)^T \mathbb{E}[g_k \mid w_k] + \frac{1}{2L} \mathbb{E}[\|g_k\|^2 \mid w_k].$$

Usando (3.3):

$$\mathbb{E}[J(w_{k+1}) \mid w_k] \leq J(w_k) - \frac{1}{L} \|\nabla J(w_k)\|^2 + \frac{1}{2L} \mathbb{E}[\|g_k\|^2 \mid w_k]. \quad (3.13)$$

Per esprimere $\mathbb{E}[\|g_k\|^2 \mid w_k]$, utilizziamo l'identità $\mathbb{E}[\|X\|^2] = \|\text{Var}(X)\|_1 + \|\mathbb{E}[X]\|^2$ (valida componente per componente). Applicandola a $X = g_k$:

$$\mathbb{E}[\|g_k\|^2 \mid w_k] = \|\text{Var}(g_k)\|_1 + \|\nabla J(w_k)\|^2. \quad (3.14)$$

Sostituendo in (3.13):

$$\mathbb{E}[J(w_{k+1}) \mid w_k] \leq J(w_k) - \frac{1}{2L} \|\nabla J(w_k)\|^2 + \frac{1}{2L} \|\text{Var}(g_k)\|_1. \quad (3.15)$$

Dalla formula per la varianza di una media campionaria (derivata in precedenza) e dall'ipotesi (3.4):

$$\|\text{Var}(g_k)\|_1 \leq \frac{\|\text{Var}(\nabla \ell(w_k; i))\|_1}{n_k} \leq \frac{\omega}{n_k}. \quad (3.16)$$

Sostituendo in (3.15):

$$\mathbb{E}[J(w_{k+1}) \mid w_k] \leq J(w_k) - \frac{1}{2L} \|\nabla J(w_k)\|^2 + \frac{\omega}{2Ln_k}. \quad (3.17)$$

Dalla λ -convessità forte di J (disuguaglianza di PL dimostrata nell'Appendice 4):

$$\|\nabla J(w_k)\|^2 \geq 2\lambda(J(w_k) - J(w_*)). \quad (3.18)$$

Sostituendo in (3.17):

$$\mathbb{E}[J(w_{k+1}) - J(w_*) \mid w_k] \leq \left(1 - \frac{\lambda}{L}\right) (J(w_k) - J(w_*)) + \frac{\omega}{2Ln_k}. \quad (3.19)$$

Prendendo il valore atteso totale e ponendo $\varepsilon_k := \mathbb{E}[J(w_k) - J(w_*)]$:

$$\varepsilon_{k+1} \leq \left(1 - \frac{\lambda}{L}\right) \varepsilon_k + \frac{\omega}{2Ln_k}. \quad (3.20)$$

Questa è l'equazione fondamentale che governa la convergenza in aspettativa.

Possiamo ora enunciare il teorema di convergenza in aspettativa.

Teorema

Teorema 3.1 (Convergenza in aspettativa). Posto $\alpha := 1 - \frac{\lambda}{L}$ e $\beta := \frac{\omega}{2L}$, per ogni $k \geq 0$ vale

$$\mathbb{E}[J(w_k) - J(w_*)] \leq \alpha^k \varepsilon_0 + \beta a \frac{\alpha^k - a^{-k}}{\alpha a - 1}, \quad (3.21)$$

dove il rapporto va inteso nel senso dei limiti se $\alpha a = 1$. Di conseguenza, posto $\rho := \max\{\alpha, 1/a\}$, se $a > 1$ si ha $\rho < 1$ e esiste una costante $C =$

$C(\varepsilon_0, \omega, \lambda, L, a)$ tale che

$$\mathbb{E}[J(w_k) - J(w_*)] \leq C\rho^k \quad \forall k \geq 0.$$

Nota

Differenza rispetto al paper (Byrd et al. 2012, Teorema 4.2). Il paper ottiene $\rho = \max\{1 - \lambda/(4L), 1/a\}$ e $C = \max\{J(w_0) - J(w_*), 2\omega/\lambda\}$ tramite un argomento induttivo che introduce un fattore $\frac{1}{2}$ aggiuntivo. La dimostrazione qui presentata ottiene invece $\rho = \max\{1 - \lambda/L, 1/a\}$ risolvendo esplicitamente la ricorrenza, il che dà un tasso deterministico $\alpha = 1 - \lambda/L$ più conservativo. Di conseguenza, l'intervallo ammissibile per a nel Corollario 3.2 è $(1, (1 - \lambda/L)^{-1}]$ anziché $(1, (1 - \lambda/(4L))^{-1}]$ come nel paper.

Dimostrazione. Iterazione della ricorrenza e forma chiusa

Dalla (3.20) possiamo esplicitare l'andamento di ε_k . Scriviamo i primi passi:

$$\begin{aligned} \varepsilon_1 &\leq \alpha\varepsilon_0 + \beta, \\ \varepsilon_2 &\leq \alpha\varepsilon_1 + \beta a^{-1} \leq \alpha(\alpha\varepsilon_0 + \beta) + \beta a^{-1} = \alpha^2\varepsilon_0 + \alpha\beta + \beta a^{-1}, \\ \varepsilon_3 &\leq \alpha\varepsilon_2 + \beta a^{-2} \leq \alpha(\alpha^2\varepsilon_0 + \alpha\beta + \beta a^{-1}) + \beta a^{-2} \\ &= \alpha^3\varepsilon_0 + \alpha^2\beta + \alpha\beta a^{-1} + \beta a^{-2}. \end{aligned}$$

Il pattern generale è

$$\varepsilon_k \leq \alpha^k \varepsilon_0 + \beta \sum_{j=0}^{k-1} \alpha^{k-1-j} a^{-j}. \quad (3.22)$$

Calcoliamo la somma geometrica finita:

$$S_k := \sum_{j=0}^{k-1} \alpha^{k-1-j} a^{-j}.$$

Ponendo $i = k - 1 - j$, si ha:

$$S_k = \sum_{i=0}^{k-1} \alpha^i a^{-(k-1-i)} = a^{-(k-1)} \sum_{i=0}^{k-1} (\alpha a)^i.$$

Se $\alpha a \neq 1$, la progressione geometrica dà:

$$S_k = a^{-(k-1)} \frac{1 - (\alpha a)^k}{1 - \alpha a} = \frac{a(\alpha^k - a^{-k})}{\alpha a - 1}. \quad (3.23)$$

Sostituendo (3.23) in (3.22) si ottiene la forma chiusa:

$$\varepsilon_k \leq \alpha^k \varepsilon_0 + \beta a \frac{\alpha^k - a^{-k}}{\alpha a - 1}, \quad (3.24)$$

che è la (3.24), cioè la (3.21) del teorema. Questa espressione è valida per $\alpha a \neq 1$; nel caso $\alpha a = 1$ si intende per limite e si ottiene $\varepsilon_k \leq (\varepsilon_0 + \beta k/\alpha) \rho^k$ (con $\rho = \alpha$), che porta allo stesso tasso esponenziale a meno di un fattore polinomiale.

Deduzione del tasso di convergenza Dalla (3.21) possiamo ricavare il tasso di convergenza studiando il comportamento asintotico. Si presentano tre casi.

- **Caso $\alpha a > 1$:** allora $\alpha > 1/a$, quindi α^k decresce più lentamente di a^{-k} ; il termine α^k domina. Trascurando a^{-k} rispetto a α^k :

$$\varepsilon_k \leq \alpha^k \varepsilon_0 + \frac{\beta a}{\alpha a - 1} \alpha^k = \left(\varepsilon_0 + \frac{\beta a}{\alpha a - 1} \right) \alpha^k.$$

Quindi $\rho = \alpha = 1 - \lambda/L$. Possiamo trascurare il fattore a^{-k} perché è un o -piccolo rispetto ad α^k : infatti $\lim_{k \rightarrow \infty} \frac{a^{-k}}{\alpha^k} = \lim_{k \rightarrow \infty} \left(\frac{1/a}{\alpha} \right)^k = \lim_{k \rightarrow \infty} \left(\frac{1}{\alpha a} \right)^k$ poiché $\alpha a > 1$, si ha $0 < \frac{1}{\alpha a} < 1 \implies \lim_{k \rightarrow \infty} \left(\frac{1}{\alpha a} \right)^k = 0 \implies \frac{a^{-k}}{\alpha^k} \rightarrow 0$

- **Caso $\alpha a < 1$:** allora $\alpha < 1/a$, quindi a^{-k} decresce più lentamente di α^k ; il termine a^{-k} domina. Trascurando α^k rispetto a a^{-k} :

$$\varepsilon_k \leq \frac{\beta a}{1 - \alpha a} a^{-k} = \frac{\beta a}{1 - \alpha a} \left(\frac{1}{a} \right)^k.$$

Quindi $\rho = 1/a$. Possiamo trascurare il fattore α^k perché è un o -piccolo rispetto a a^{-k} : infatti $\lim_{k \rightarrow \infty} \frac{\alpha^k}{a^{-k}} = \lim_{k \rightarrow \infty} \frac{\alpha^k}{(1/a)^k} = \lim_{k \rightarrow \infty} (\alpha a)^k$ poiché $0 < \alpha a < 1$, si ha $0 < \alpha a < 1 \implies \lim_{k \rightarrow \infty} (\alpha a)^k = 0 \implies \frac{\alpha^k}{a^{-k}} \rightarrow 0$.

- **Caso $\alpha a = 1$:** allora $\alpha = 1/a$. La somma geometrica si riduce a $S_k = k a^{-(k-1)}$, da cui:

$$\varepsilon_k \leq \alpha^k \varepsilon_0 + \beta k a^{-(k-1)} = (\varepsilon_0 + \beta k / \alpha) \rho^k, \quad \rho := \alpha = 1/a.$$

Poiché per ogni $\rho' \in (\rho, 1)$ esiste una costante C (indipendente da k) tale che $k \rho^k \leq C(\rho')^k$, la tesi vale con tasso ρ' .

Quindi per $\alpha a > 1$ si ottiene $\rho = \alpha$ e $C = \varepsilon_0 + \frac{\beta a}{\alpha a - 1}$; se $\alpha a < 1$ si ottiene $\rho = 1/a$ e $C = \varepsilon_0 + \frac{\beta a}{1 - \alpha a}$ (o, più strettamente, $C = \max\{\varepsilon_0, \frac{\beta a}{1 - \alpha a}\}$).

In tutti i casi si ha:

$$\varepsilon_k \leq C \rho^k, \quad \rho := \max\{\alpha, 1/a\}, \quad (3.25)$$

per una opportuna costante $C = C(\varepsilon_0, \omega, \lambda, L, a)$.

□

Corollario sulla complessità totale

Corollario

Corollario 3.2 (Complessità totale). Supponiamo $n_k = \lceil a^k \rceil$ con

$$a \in \left(1, \underbrace{\left(1 - \frac{\lambda}{L} \right)^{-1}}_{1/\alpha} \right],$$

e che (3.4) valga per tutti i w_k . Allora il numero totale di valutazioni dei

gradienti $\nabla \ell(w; i)$ necessarie per ottenere $\mathbb{E}[J(w_k) - J(w_*)] \leq \varepsilon$ è

$$O\left(\frac{L}{\lambda \varepsilon}\right), \quad (3.26)$$

e il lavoro totale dell'algoritmo (3.10) con campionamento dinamico (3.9) è

$$\mathcal{T}_{\text{tot}} = O\left(\frac{Lm}{\lambda \varepsilon} \max\left\{J(w_0) - J(w_*), \frac{\omega}{\lambda}\right\}\right), \quad (3.27)$$

dove m è il numero di variabili.

Dimostrazione. Con la scelta $a \leq \alpha^{-1}$, si ha $\rho = 1/a$. Dalla (3.21), il numero di iterazioni k necessario per ottenere $\mathbb{E}[J(w_k) - J(w_*)] \leq \varepsilon$ soddisfa $a^k \geq C/\varepsilon$. Poiché $\rho = 1/a$, la condizione $C\rho^k \leq \varepsilon$ equivale a $a^k \geq C/\varepsilon$. Il numero di iterazioni necessario soddisfa quindi $a^k = \Theta(C/\varepsilon)$. Il costo in valutazioni del gradiente è:

$$\sum_{j=0}^{k-1} n_j \leq 2 \sum_{j=0}^{k-1} a^j = 2 \cdot \frac{a^k - 1}{a - 1} \leq \frac{2}{a - 1} a^k = O\left(\frac{1}{\varepsilon}\right).$$

Moltiplicando per il costo $O(m)$ di ogni gradiente si ottiene (3.27). $\square$

Scelta di a e confronto Il risultato (3.27) vale se a è scelto nell'intervallo

$$a \in \left(1, \left(1 - \frac{\lambda}{L}\right)^{-1}\right].$$

Non è necessario conoscere il numero di condizionamento $\kappa = L/\lambda$ esattamente; qualsiasi sovrastima va bene:

$$a = \left(1 - \frac{1}{\kappa'}\right)^{-1}, \quad \kappa' \geq \kappa. \quad (3.28)$$

Dalla (3.27) ignorando la dipendenza dal punto iniziale (come in [2]) e assumendo il regime $\omega/\lambda \geq J(w_0) - J(w_*)$, si ottiene $\mathcal{T}_{\text{tot}} = O(m\kappa\omega/\lambda\varepsilon)$, che è il bound riportato in tabella sotto.

Confrontiamo questo risultato con i bound di complessità noti per il metodo a gradiente a campione fisso e il metodo del gradiente stocastico (SGD), calcolati da Bottou e Bousquet [2]: nella tabella, lo scalare $\bar{\alpha} \in [1/2, 1]$ è il *tasso di stima* del metodo a campione fisso, come definito in [2].

Nome algoritmo	Bound
Metodo a gradiente dinamico (questo lavoro)	$O(m\kappa\omega/\lambda\varepsilon)$
Metodo a gradiente a campione fisso	$O(m^2\kappa\varepsilon^{-1/\bar{\alpha}}\log^2(1/\varepsilon))$
Metodo del gradiente stocastico	$O(m\bar{\nu}\kappa^2/\varepsilon)$

Tabella 1: Bound di complessità per tre metodi. Qui m è il numero di variabili, $\kappa = L/\lambda$, ω e $\bar{\nu}$ sono definiti in (3.4) e (3.29), e $\bar{\alpha} \in [1/2, 1]$.

I metodi a campione fisso e stocastico sono definiti come:

- Metodo a campione fisso: $w_{k+1} = w_k - \frac{1}{L} \frac{\sum_{i=1}^{N_\varepsilon} \nabla \ell(w_k; i)}{N_\varepsilon},$
- Metodo del gradiente stocastico: $w_{k+1} = w_k - \frac{1}{\lambda k} \nabla \ell(w_k; i_k),$

dove N_ε specifica una dimensione del campione tale che $\mathbb{E}[|J(w_*) - J_S(w_*)|] < \varepsilon$ vale in aspettativa. I bound di complessità presentati in [2] ignorano l'effetto del punto iniziale w_0 .

Lo scalare $\bar{\alpha} \in [1/2, 1]$, chiamato *tasso di stima* [2], dipende dalle proprietà della funzione di perdita; la costante $\bar{\nu}$ è definita come

$$\bar{\nu} = \text{trace}(H^{-1}G), \quad H = \nabla^2 J(w_*), \quad G = \mathbb{E}[\nabla \ell(w_*; i) \nabla \ell(w_*; i)^T]. \quad (3.29)$$

Poiché $H \succeq \lambda I$ e ω è un bound per $\|\text{Var}(\nabla \ell(w_k; I))\|_1$, si ha $\bar{\nu} \approx \omega/\lambda$. Sostituendo nel bound per SGD:

$$O(m\bar{\nu}\kappa^2/\varepsilon) = O\left(\frac{m\omega\kappa^2}{\lambda\varepsilon}\right). \quad (3.30)$$

Il bound per il metodo dinamico (3.27) è invece $O(m\kappa\omega/(\lambda\varepsilon))$. La differenza principale è il termine κ^2 in SGD contro κ nel metodo dinamico. Concludiamo che **il metodo a gradiente dinamico gode di un bound di complessità competitivo con quello del metodo del gradiente stocastico**, risultando particolarmente vantaggioso per problemi mal condizionati (grande κ).

La strategia dinamica $n_k = \lceil a^k \rceil$ con a scelto in base a una stima (bound superiore) di κ garantisce un compromesso ottimale tra costo per iterazione e velocità di convergenza. In pratica, la stima di κ può essere sostituita da una regola euristica semplice (ad esempio, $a = 1.1$), e l'algoritmo si adatta automaticamente.

2.2 Metodo di Newton-CG con Campionamento Dinamico

Il metodo del gradiente (steepest descent) può convergere lentamente, specialmente su problemi mal condizionati; gli algoritmi che incorporano informazione di curvatura (secondo ordine) sulla funzione obiettivo possono essere molto più efficienti. In questa sezione descriviamo un metodo di Newton-CG (gradiente coniugato) *Hessian-free* (non costruiamo e non memorizziamo esplicitamente la matrice Hessiana di $J(w)$, ma ne usiamo solo i prodotti per un vettore), che impiega tecniche di campionamento dinamico sia per il calcolo della funzione e del gradiente, sia per i prodotti Hessiana-vettore.

2.2.1 Struttura del metodo

Il metodo estende l'algoritmo del gradiente a campione dinamico (quello pratico, non la sua variante idealizzata con $n_k = \lceil a^k \rceil$). Rispetto ai metodi che utilizzano un campione fisso per il gradiente e un numero fisso di iterazioni del CG, questo metodo presenta due innovazioni principali:

1. Il campione $\mathcal{S}_k$ per il gradiente non è più fisso; qui si incorporano le tecniche di campionamento dinamico dell'algoritmo visto precedentemente.
2. Il numero di iterazioni del gradiente coniugato non è più fisso; si propone un criterio di terminazione automatico per il CG, che si adatta alla dimensione del campione e all'accuratezza dell'approssimazione dell'Hessiana.

Ad ogni iterazione, il metodo sceglie due campioni $\mathcal{S}_k$ e $\mathcal{H}_k$ tali che

$$|\mathcal{H}_k| \ll |\mathcal{S}_k|,$$

e definisce la direzione di ricerca d_k come soluzione approssimata del sistema lineare

$$\underbrace{\nabla^2 J_{\mathcal{H}_k}(w_k) d = -\nabla J_{\mathcal{S}_k}(w_k)}_{\text{(metodo di Newton classico)}}. \quad (4.1)$$

L'uso di due campioni distinti è cruciale:

- $\mathcal{S}_k$ per il gradiente $\nabla J_{\mathcal{S}_k}(w_k)$, per garantire una direzione di discesa affidabile.
- $\mathcal{H}_k$ per l'Hessiana $\nabla^2 J_{\mathcal{H}_k}(w_k)$, per mantenere il costo dei prodotti Hessiana-vettore sufficientemente basso.

Il sistema (4.1) viene risolto con il **metodo del gradiente coniugato (CG)** **Hessian-free**: la matrice $\nabla^2 J_{\mathcal{H}_k}(w_k)$ non viene mai costruita esplicitamente; invece, si calcola direttamente il prodotto di questa Hessiana per un vettore.

Il rapporto

$$R = \frac{|\mathcal{H}_k|}{|\mathcal{S}_k|} < 1 \quad (4.2)$$

viene fissato a priori ed è mantenuto costante durante tutto l'algoritmo. Quando il campione per il gradiente $\mathcal{S}_k$ cresce (dinamicamente), la dimensione di $\mathcal{H}_k$ viene aggiornata come $|\mathcal{H}_k| = \lceil R \cdot |\mathcal{S}_k| \rceil$, in modo che $\mathcal{H}_k$ cresca proporzionalmente a $\mathcal{S}_k$. Una linea guida pratica è che il costo totale dei prodotti Hessiana-vettore nel CG non superi il costo di una valutazione del gradiente $\nabla J_{\mathcal{S}_k}$.

2.2.2 Criterio di terminazione del gradiente coniugato

Proponiamo un criterio automatico per decidere con quale accuratezza risolvere il sistema (4.1). L'idea chiave è che il *residuo* del CG,

$$r_k \stackrel{\text{def}}{=} \nabla^2 J_{\mathcal{H}_k}(w_k)d + \nabla J_{\mathcal{S}_k}(w_k), \quad (4.3)$$

non deve essere più piccolo dell'*errore di approssimazione dell'Hessiana*.

Per comprendere questo punto, scriviamo il residuo del sistema di Newton che userebbe il campione grande $\mathcal{S}_k$ sia per il gradiente che per l'Hessiana:

$$\underbrace{\nabla^2 J_{\mathcal{S}_k}(w_k)d + \nabla J_{\mathcal{S}_k}(w_k)}_{\text{Residuo del sistema}} = \underbrace{r_k}_{\text{Residuo del CG}} + \underbrace{[\nabla^2 J_{\mathcal{S}_k}(w_k) - \nabla^2 J_{\mathcal{H}_k}(w_k)] d}_{\text{Errore di Hessiana} = \Delta_{\mathcal{H}_k}(w_k; d)}. \quad (4.4)$$

Il termine $\Delta_{\mathcal{H}_k}(w_k; d)$ misura l'errore nell'approssimazione dell'Hessiana lungo la direzione d , dovuto all'uso del campione più piccolo $\mathcal{H}_k$ e questo è costante durante l'algoritmo. È chiaro dall'equazione sopra che se l'errore di Hessiana $\Delta_{\mathcal{H}_k}$ non si può eliminare, non serve ridurre r_k fino a zero. Basta ridurlo fino a quando è comparabile a $\Delta_{\mathcal{H}_k}$. Oltre quel punto, il tempo speso è sprecato.

Il nostro obiettivo è quindi bilanciare i due termini al membro destro di (4.4), terminando il CG quando il residuo r_k è dello stesso ordine dell'errore dell'hessiana.

Tuttavia, $\Delta_{\mathcal{H}_k}(w; d)$, analogamente all'errore di approssimazione del gradiente discusso precedentemente, non è direttamente calcolabile, perché richiederebbe di calcolare $\nabla^2 J_{\mathcal{S}_k}(w_k)d$ (cioè il prodotto Hessiana-vettore sul campione grande), il che vanificherebbe il vantaggio del sottocampionamento dell'Hessiana.

Seguendo la stessa strategia implementata precedentemente per il gradiente, usiamo la varianza campionaria del prodotto Hessiana-vettore per stimare l'errore.

Errore del gradiente (sez. 3.2.2)	Errore dell'Hessiana (sez. 4.2.2)
$e_k = g_k - \nabla J(w_k)$	$\Delta_{\mathcal{H}_k}(w_k; d) = [\nabla^2 J_{\mathcal{S}_k}(w_k) - \nabla^2 J_{\mathcal{H}_k}(w_k)] d$
Non calcolabile perché richiederebbe $\nabla J(w_k)$ su tutti i dati	Non calcolabile perché richiederebbe $\nabla^2 J_{\mathcal{S}_k}(w_k)d$ su tutto il campione grande
Si aggira il problema stimando $\mathbb{E}[\ e_k\ ^2]$ con la varianza campionaria dei gradienti individuali $\nabla \ell(w_k; i)$ sul batch	Si aggira il problema stimando $\mathbb{E}[\ \Delta\ ^2]$ con la varianza campionaria dei prodotti Hessiana-vettore individuali $\nabla^2 \ell(w_k; i)d$ sul sottocampione $\mathcal{H}_k$

Per comprendere la derivazione, partiamo dalla definizione di $\Delta_{\mathcal{H}_k}$.

Precedentemente avevamo definito

$$\Delta_{\mathcal{H}_k}(w_k; d) \stackrel{\text{def}}{=} [\nabla^2 J_{\mathcal{S}_k}(w_k) - \nabla^2 J_{\mathcal{H}_k}(w_k)] d.$$

Sviluppando le medie campionarie e ponendo $X_i = \nabla^2 \ell(w_k; i)d$, abbiamo

$$\nabla^2 J_{\mathcal{S}_k}(w_k)d = \frac{1}{|\mathcal{S}_k|} \sum_{i \in \mathcal{S}_k} X_i := \mu_{\mathcal{S}} \quad \text{e} \quad \nabla^2 J_{\mathcal{H}_k}(w_k)d = \frac{1}{|\mathcal{H}_k|} \sum_{i \in \mathcal{H}_k} X_i := \mu_{\mathcal{H}}.$$

Indichiamo $\Delta_{\mathcal{H}_k}(w_k; d)$ con Δ , poiché $\mathcal{H}_k$ è un sottocampione casuale estratto senza reinserimento dall'insieme $\mathcal{S}_k$, la quantità $\mu_{\mathcal{H}}$ è uno stimatore della media della

popolazione finita $\mu_{\mathcal{S}}$. L'errore di stima è $\mu_{\mathcal{H}} - \mu_{\mathcal{S}}$, il cui quadrato atteso è la varianza dello stimatore $\mu_{\mathcal{H}}$. Osserviamo che

$$\|\Delta\|^2 = \|\mu_{\mathcal{S}} - \mu_{\mathcal{H}}\|^2 = \|\mu_{\mathcal{H}} - \mu_{\mathcal{S}}\|^2 \implies \mathbb{E}[\|\Delta\|^2] = \mathbb{E}[\|\mu_{\mathcal{H}} - \mu_{\mathcal{S}}\|^2] = \text{Var}(\mu_{\mathcal{H}}).$$

A questo punto, usiamo la formula per la varianza di una media campionaria in un campionamento senza reinserimento che avevamo derivato precedentemente. Consideriamo una popolazione finita di dimensione $N = |\mathcal{S}_k|$ con elementi $X_1, \dots, X_N$, e un campione senza reinserimento di dimensione $n = |\mathcal{H}_k|$.

$$\text{Var}(\mu_{\mathcal{H}}) = \frac{\sigma^2}{n} \cdot \frac{N - n}{N - 1}.$$

Questa è la formula classica per la varianza di una media campionaria in un campionamento senza reinserimento, che include il fattore di correzione per popolazione finita $\frac{N-n}{N-1}$. Sostituendo $n = |\mathcal{H}_k|$, $N = |\mathcal{S}_k|$ e prendendo la norma $\|\cdot\|_1$ componente per componente (poiché gli X_i sono vettori), otteniamo:

$$\mathbb{E}[\|\Delta\|^2] = \frac{\overbrace{\left\| \frac{1}{|\mathcal{S}_k|} \sum_{i \in \mathcal{S}_k} (X_i - \mu_{\mathcal{S}})^2 \right\|_1}^{\sigma^2}}{|\mathcal{H}_k|} \cdot \frac{|\mathcal{S}_k| - |\mathcal{H}_k|}{|\mathcal{S}_k| - 1}.$$

Ritornando alla notazione originale, $X_i = \nabla^2 \ell(w_k; i) d$, e ricordando che σ^2 denota la varianza della popolazione $\mathcal{S}_k$ che approssimiamo con la varianza campionaria calcolata su $\mathcal{H}_k$, otteniamo la prima approssimazione:

$$\mathbb{E}[\|\Delta_{\mathcal{H}_k}(w_k; d)\|_2^2] \approx \frac{\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) d)\|_1 (|\mathcal{S}_k| - |\mathcal{H}_k|)}{|\mathcal{H}_k| (|\mathcal{S}_k| - 1)}. \quad (4.5a)$$

Questo viene fatto siccome l'uso di σ^2 richiederebbe di calcolare il prodotto Hessiana-vettore su tutto il campione grande $\mathcal{S}_k$, il che vanificherebbe il vantaggio del sotto-campionamento dell'Hessiana.

Poiché nel paper si assume $|\mathcal{H}_k| \ll |\mathcal{S}_k|$ (il campione per l'Hessiana è molto più piccolo di quello per il gradiente), il rapporto $\frac{|\mathcal{S}_k| - |\mathcal{H}_k|}{|\mathcal{S}_k| - 1}$ è circa uguale a 1. Si ottiene quindi la semplificazione finale:

$$\mathbb{E}[\Delta_{\mathcal{H}_k}(w_k; d)^2] \approx \frac{\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) d)\|_1}{|\mathcal{H}_k|}. \quad (4.5b)$$

Questa equazione fornisce una stima dell'errore di approssimazione dell'Hessiana per un generico vettore d . Osserviamo che la mappa $d \mapsto \nabla^2 \ell(w_k; i) d$ è *lineare* in d . Di conseguenza, la sua varianza (calcolata su un campione fissato) è una *forma quadratica* in d : esiste una matrice di covarianza campionaria $\Sigma_{\mathcal{H}_k} \in \mathbb{R}^{m \times m}$, simmetrica semidefinita positiva, tale che

$$\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) d) = d^T \Sigma_{\mathcal{H}_k} d \in \mathbb{R}^m,$$

dove il membro sinistro è inteso componente per componente e la relazione vale per ogni componente $j = 1, \dots, m$ con la corrispondente matrice $\Sigma_{\mathcal{H}_k}^{(j)}$. Questa forma quadratica gode della proprietà di omogeneità

$$\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i)(\lambda d)) = \lambda^2 \text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) d).$$

All'inizio dell'algoritmo, la soluzione candidata è $d_0 = 0$ e il residuo iniziale è $r_0 = \nabla J_{S_k}(w_k)$. La prima direzione di ricerca del CG è quindi $p_0 = -r_0$. Per evitare di ricalcolare la varianza a ogni iterazione del CG (operazione costosa), il paper propone di calcolarla *una sola volta* per $d = p_0$. Sfruttando l'omogeneità della forma quadratica, si definisce

$$\alpha := \frac{\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) p_0)\|_1}{\|p_0\|_2^2} = \frac{\|p_0^T \Sigma_{\mathcal{H}_k} p_0\|_1}{\|p_0\|_2^2}. \quad (4.6)$$

Poiché la varianza effettiva per una generica direzione d_j è $\|d_j^T \Sigma_{\mathcal{H}_k} d_j\|_1$, e non $\alpha \|d_j\|^2$, l'uso di α per tutte le direzioni costituisce un'approssimazione, α misura l'influenza media della matrice di covarianza campionaria dei prodotti Hessiana-vettore su un vettore generico, è una media pesata degli autovalori di $\Sigma_{\mathcal{H}_k}$ ed è sempre compreso tra $\lambda_{\min}(\Sigma_{\mathcal{H}_k})$ e $\lambda_{\max}(\Sigma_{\mathcal{H}_k})$. Il metodo assume che la forma quadratica sia sufficientemente ben condizionata perché la dipendenza angolare sia trascurabile ai fini del test di arresto; ciò giustifica la stima

$$\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) d_j)\|_1 \approx \alpha \|d_j\|_2^2 = \frac{\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) p_0)\|_1}{\|p_0\|_2^2} \|d_j\|_2^2. \quad (4.7)$$

Ora, dalla (4.5b), sostituendo la varianza stimata tramite la (4.7), definiamo

$$\gamma := \frac{\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) p_0)\|_1}{|\mathcal{H}_k| \|p_0\|_2^2}, \quad \Psi(d) := \gamma \|d\|_2^2. \quad (4.8)$$

Test di arresto del CG. La soglia $\Psi(d_j)$ definita in (4.8) viene utilizzata per decidere quando fermare il metodo del gradiente coniugato. Ad ogni iterazione j del CG, si controlla se il residuo r_{j+1} soddisfa

$$\|r_{j+1}\|_2^2 \leq \Psi(d_j) = \gamma \|d_j\|_2^2. \quad (4.9)$$

Se la condizione è verificata, il CG termina e la soluzione corrente d_{j+1} viene accettata come direzione di Newton approssimata. In caso contrario, si prosegue con la successiva iterazione del CG. Questo criterio di arresto è particolarmente efficace perché evita calcoli inutili quando l'approssimazione dell'Hessiana è limitata dal campione $\mathcal{H}_k$; si adatta alla distanza dal minimo poiché $\Psi \propto \|r_0\|^2$; previene instabilità numeriche perché Ψ cresce con $\|d_j\|^2$, fermando il CG prima che la direzione diventi eccessivamente grande.

L'algoritmo quindi calcola γ una volta sola all'inizio del CG, e poi ad ogni iterazione aggiorna $\Psi(d_j) = \gamma \|d_j\|^2$ e verifica la condizione (4.9).

2.2.3 Pseudocodice (Algoritmo Newton-CG)

Algoritmo

Newton-CG con Campionamento Dinamico

Input: w_0 (punto iniziale), $\mathcal{S}_0$ (batch iniziale per gradiente), $\mathcal{H}_0 \subset \mathcal{S}_0$ (batch iniziale per Hessiana), $R = |\mathcal{H}_0|/|\mathcal{S}_0|$ (rapporto costante), $\theta \in (0, 1)$ (tolleranza per varianza del gradiente), $c_1, c_2 \in (0, 1)$ con $c_1 < c_2$ (parametri Wolfe).

Inizializzazione: poni $k = 0$.

Ripeti per ogni iterazione k fino a convergenza:

1. **Calcola la direzione di Newton approssimata d_k mediante gradiente coniugato (CG):**

- (a) $d_0 = 0, r_0 = \nabla J_{\mathcal{S}_k}(w_k), p_0 = -r_0, j = 0, \Psi = 0.$

- (b) $\gamma = \frac{\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) p_0)\|_1}{|\mathcal{H}_k| \|p_0\|_2^2}.$

- (c) Finchè $\|r_j\|_2^2 > \Psi$:

- i. $s_j = \nabla^2 J_{\mathcal{H}_k}(w_k) p_j$

- ii. $\alpha_j = \frac{r_j^T r_j}{p_j^T s_j}.$

- iii. $d_{j+1} = d_j + \alpha_j p_j, r_{j+1} = r_j + \alpha_j s_j.$

- iv. $\beta_{j+1} = \frac{r_{j+1}^T r_{j+1}}{r_j^T r_j}.$

- v. $p_{j+1} = -r_{j+1} + \beta_{j+1} p_j.$

- vi. $j = j + 1, \Psi = \gamma \|d_j\|_2^2.$

- (d) $d_k = d_j$

2. **Line search con condizioni di Wolfe:** Trova $\alpha_k > 0$ tale che:

$$J_{\mathcal{S}_k}(w_k + \alpha_k d_k) \leq J_{\mathcal{S}_k}(w_k) + c_1 \alpha_k \nabla J_{\mathcal{S}_k}(w_k)^T d_k,$$

$$\nabla J_{\mathcal{S}_k}(w_k + \alpha_k d_k)^T d_k \geq c_2 \nabla J_{\mathcal{S}_k}(w_k)^T d_k.$$

3. Aggiorna il punto: $w_{k+1} = w_k + \alpha_k d_k.$

4. **Aggiornamento dinamico del batch per il gradiente:**

- (a) Seleziona un nuovo campione $\mathcal{S}_{k+1}$ della stessa dimensione di $\mathcal{S}_k.$

- (b) Calcola la varianza campionaria $\hat{\mathcal{V}} = \text{Var}_{i \in \mathcal{S}_{k+1}}(\nabla \ell(w_{k+1}; i)).$

- (c) Se $\frac{\|\hat{\mathcal{V}}\|_1}{|\mathcal{S}_{k+1}|} > \theta^2 \|\nabla J_{\mathcal{S}_{k+1}}(w_{k+1})\|_2^2$, aumenta il batch:

$$|\mathcal{S}_{k+1}| \leftarrow \left\lceil \frac{\|\hat{\mathcal{V}}\|_1}{\theta^2 \|\nabla J_{\mathcal{S}_{k+1}}(w_{k+1})\|_2^2} \right\rceil.$$

5. Seleziona $\mathcal{H}_{k+1} \subseteq \mathcal{S}_{k+1}$ tale che $|\mathcal{H}_{k+1}| = R \cdot |\mathcal{S}_{k+1}|.$

6. Incrementa $k \leftarrow k + 1.$

Nota Tecnica

L'algoritmo del gradiente coniugato serve a minimizzare l'errore di $x \approx \bar{x}$ t.c. $A\bar{x} = b$. Nel contesto del metodo di Newton, i suoi elementi vengono reinterpretati come segue:

- $A = H_k = \nabla^2 J(w_k)$ (Hessiana della funzione obiettivo, calcolata sul campione $\mathcal{H}_k$);
- $b = -g_k = -\nabla J(w_k)$ (gradiente negativo, calcolato sul campione $\mathcal{S}_k$);
- $x = d_k$ (direzione di Newton, che risolve $H_k d_k = -g_k$).

Il CG, quindi, viene usato internamente per approssimare la direzione di Newton senza costruire esplicitamente l'Hessiana.

INPUT: $A \in \mathbb{R}^{n \times n}$ (simmetrica def. pos.), $b \in \mathbb{R}^n$, $x_0 = \mathbf{1}$, $\text{tol} = 10^{-10} \|b\|_2$, $it_{\max} = 1000$

- Calcola $r = b - Ax_0$, $p = r$, residui = $[\|r\|_2]$
- Per $k = 1$ a $it_{\max}$:
 - $Ap = A \cdot p$
 - $\alpha = \frac{r^T p}{p^T Ap}$ (Passo ottimale)
 - $x = x + \alpha p$
 - $r' = r - \alpha Ap$
 - $\beta = \frac{r'^T r'}{r^T r}$
 - $p = r' + \beta p$
 - $r = r'$
 - residui = $[\text{residui}, \|r\|_2]$
 - Se $\|r\|_2 \leq \text{tol}$: termina

OUTPUT: x , residui

Questo metodo usa un passo α_k che si ottiene minimizzando il funzionale

$$\phi(x) = \frac{1}{2} x^T A x - b^T x$$

per $x = x_k + \alpha p_k$, annullando la derivata di tale funzionale rispetto ad α :

$$\left. \frac{d}{d\alpha} \phi(x_k + \alpha p_k) \right|_{\alpha=\alpha_k} = (Ax_k - b)^T p_k + \alpha_k p_k^T A p_k = 0.$$

Poiché $Ax_k - b = -r_k$, dove $r_k = b - Ax_k$ è il residuo, si ha:

$$-r_k^T p_k + \alpha_k p_k^T A p_k = 0 \implies \alpha_k = \frac{r_k^T p_k}{p_k^T A p_k}. \quad (4.10)$$

Ora però si può dimostrare che la formula usata in questo metodo è equivalente a questa. La dimostrazione completa è riportata nell'Appendice 4.3. In sintesi, dalla condizione di stazionarietà al passo precedente e dall'aggiornamento del residuo si ottiene $r_k^T p_{k-1} = 0$, da cui segue $r_k^T p_k = r_k^T r_k$ e quindi l'equivalenza delle due formule. La formula con $r_k^T r_k$ è preferita nelle implementazioni numeriche perché è più stabile, poiché non dipende dall'ortogonalità numerica di $r_k^T p_{k-1}$, che in presenza di errori di arrotondamento non è esattamente zero.

In sostanza l'algoritmo a ogni iterazione k estrae due campioni distinti dal dataset. Un campione S_k grande per calcolare il gradiente $g_k = \nabla J_{S_k}(w_k)$, perché deve essere una stima accurata della direzione di discesa, e un sottocampione H_k con $|H_k| = R \cdot |S_k|$ e $R \ll 1$ per calcolare l'Hessiana $H_k = \nabla^2 J_{H_k}(w_k)$, poiché i prodotti Hessiana-vettore sono costosi e una stima della curvatura basta. Per aggiornare i pesi usa la direzione di Newton che si ottiene minimizzando l'espansione di Taylor per $J(w)$ al secondo ordine attorno a w_k : $J(w_k + d) \approx \tilde{J}(d) = J(w_k) + \nabla J(w_k)^T d + \frac{1}{2} d^T \nabla^2 J(w_k) d$, dove w_k è il punto corrente e d è la direzione. Troviamo la d che minimizza questo modello quadratico annullandone il gradiente rispetto a d , ottenendo il sistema lineare $\nabla^2 J(w_k) d = -\nabla J(w_k)$, che è praticamente del tipo $Ax = b$ con $A = \nabla^2 J(w_k)$ e $b = -\nabla J(w_k)$. Per risolvere questo sistema non si inverte mai la matrice (troppo costoso), ma si usa il metodo del gradiente coniugato Hessian-free, facendo solo prodotti $H_k \cdot v$ senza costruire H_k .

Ora poiché l'Hessiana è calcolata su un campione piccolo H_k e quindi è approssimata, non ha senso risolvere il sistema con precisione assoluta. Il CG si ferma quando il suo residuo è confrontabile con l'errore di approssimazione dell'Hessiana, stimato dalla varianza campionaria dei prodotti $\nabla^2 \ell(w_k; i) \cdot p_0$ su H_k . In questo modo non si spreca iterazioni del CG a ridurre un errore che è già dominato dal rumore del campionamento.

Infine si esegue una line search di Armijo lungo la direzione d_k trovata, si aggiorna $w_{k+1} = w_k + \alpha_k d_k$, e come nell'algoritmo visto precedentemente si verifica la condizione di controllo della varianza (CCV) sul gradiente: se la varianza campionaria di g_k è troppo alta rispetto a $\|g_k\|^2$, si aumenta la dimensione del batch n_k per l'iterazione successiva, in modo da avere più precisione man mano che ci si avvicina al minimo.

Ora il metodo del gradiente coniugato è un metodo numerico per trovare x che approssimi la soluzione di $Ax = b$, ovvero minimizzi il funzionale $\phi(x) = \frac{1}{2} x^T A x - b^T x$ e per farlo cerca di minimizzare $\|e^{k+1}\|_A^2$ cercando in $e^k + \text{span}(p^k, p^{k-1})$; equivalentemente pone $e^{k+1} \perp_A p^k$ ed $e^{k+1} \perp_A p^{k-1}$, dalla prima condizione troviamo α_k e dalla seconda β_k (Dimostrazione nell'appendice).

2.3 Metodo Newton-CG per Modelli con Regularizzazione L_1

2.3.1 Formulazione del Problema

Nei modelli di apprendimento statistico con un numero molto elevato di parametri, l'aggiunta di un termine di regularizzazione L_1 è particolarmente vantaggiosa perché produce soluzioni *sparse*: molti coefficienti vengono automaticamente portati a zero, semplificando il modello e migliorandone l'interpretabilità.

La funzione obiettivo (1) viene quindi modificata aggiungendo un termine di penalità sulla norma L_1 dei parametri:

$$F(w) = \frac{1}{N} \sum_{i=1}^N \ell(f(w; x_i), y_i) + \nu \|w\|_1 = J(w) + \nu \|w\|_1, \quad (6.1)$$

dove:

- $\nu > 0$ è un parametro di penalità (fisso);
- $\|w\|_1 = \sum_{j=1}^m |w_j|$ è la norma L_1 del vettore dei parametri;
- $J(w)$ è la funzione di perdita empirica definita in (1).

Nel contesto mini-batch, si sceglie un sottoinsieme $\mathcal{S} \subseteq \{1, \dots, N\}$ del training set e si risolve il problema approssimato:

$$\min_{w \in \mathbb{R}^m} F_{\mathcal{S}}(w) = J_{\mathcal{S}}(w) + \nu \|w\|_1, \quad (6.2)$$

dove $J_{\mathcal{S}}$ è definita come in (3.1).

2.3.2 Il gradiente generalizzato

Per comprendere come si costruisce il gradiente generalizzato di una funzione contenente un termine L_1 , è utile partire dal caso più semplice in una variabile.

Caso 1D: la funzione $\nu|x|$. Consideriamo la funzione $\phi(x) = \nu|x|$, con $x \in \mathbb{R}$, $\nu \in \mathbb{R}$, $\nu > 0$. Questa funzione presenta un punto di non derivabilità in $x = 0$:

- se $x > 0$, la derivata è $\phi'(x) = +\nu$;
- se $x < 0$, la derivata è $\phi'(x) = -\nu$;
- se $x = 0$, la derivata classica non esiste.

Siccome la derivata classica non esiste, si utilizza il concetto di *subgradiente* (o gradiente generalizzato). Per $\phi(x) = \nu|x|$, il subgradiente in $x = 0$ è l'intervallo:

$$\partial\phi(0) = [-\nu, +\nu].$$

Questo significa che in $x = 0$ possiamo scegliere qualsiasi valore di pendenza compreso tra $-\nu$ e $+\nu$, a seconda della direzione di discesa che vogliamo seguire.

Estensione al caso multidimensionale. Consideriamo ora:

$$F(w) = \underbrace{J(w)}_{\text{parte liscia}} + \underbrace{\nu \sum_{i=1}^m |w_i|}_{\text{parte non liscia}}.$$

La parte non liscia è una somma di termini $\nu|w_i|$, uno per ogni componente di w . Poiché la somma si decompone componente per componente, possiamo calcolare il gradiente generalizzato di F separatamente per ogni coordinata.

Per la i -esima componente, abbiamo due contributi:

1. la derivata parziale della parte liscia: $\frac{\partial J(w)}{\partial w_i}$;
2. il subgradiente di $\nu|w_i|$:

$$\partial(\nu|w_i|) = \begin{cases} +\nu & \text{se } w_i > 0, \\ -\nu & \text{se } w_i < 0, \\ [-\nu, +\nu] & \text{se } w_i = 0. \end{cases}$$

Pertanto, per ogni componente i , il gradiente generalizzato $\tilde{\nabla}F(w)$ è dato da:

$$[\tilde{\nabla}F(w)]_i = \underbrace{\frac{\partial J(w)}{\partial w_i}}_{\text{gradiente di } J} + \underbrace{g_i}_{\text{subgradiente di } \nu|w_i|} \quad g_i \in \begin{cases} \{+\nu\} & \text{se } w_i > 0, \\ \{-\nu\} & \text{se } w_i < 0, \\ [-\nu, +\nu] & \text{se } w_i = 0. \end{cases}$$

Quando $w_i = 0$, abbiamo libertà di scegliere g_i in $[-\nu, +\nu]$. La scelta viene effettuata in modo che in w^* si abbia $\tilde{\nabla}F(w^*) = 0$. Per ogni coordinata i , definiamo:

$$\begin{cases} d_i := \frac{\partial J}{\partial w_i} \\ g_i \in [-\nu, +\nu] \end{cases} \implies [\tilde{\nabla}F]_i = d_i + g_i.$$

La scelta di g_i viene effettuata risolvendo il problema di proiezione:

$$g_i = \arg \min_{g \in [-\nu, \nu]} |d_i + g|,$$

cioè si sceglie il subgradiente che rende il gradiente generalizzato $d_i + g_i$ il più vicino possibile a zero. Ne derivano i tre casi seguenti:

Condizione su $d_i = \partial J / \partial w_i$	Scelta di g_i	Risultato $[\tilde{\nabla}F]_i$	Effetto
$d_i < -\nu$	$g_i = +\nu$	$d_i + \nu < 0$	$\tilde{\nabla}F < 0$: la discesa spinge w_i verso destra (aumenta)
$d_i > +\nu$	$g_i = -\nu$	$d_i - \nu > 0$	$\tilde{\nabla}F > 0$: la discesa spinge w_i verso sinistra (diminuisce)
$-\nu \leq d_i \leq +\nu$	$g_i = -d_i$	$d_i + g_i = 0$	$\tilde{\nabla}F = 0$: $w_i = 0$ è punto stazionario (minimo)

In sintesi, la regola di proiezione $g_i = \arg \min_{g \in [-\nu, \nu]} |d_i + g|$ garantisce che, nel punto di minimo w^* , si abbia esattamente $\tilde{\nabla} F(w^*) = 0$ per tutte le coordinate in cui $|d_i| \leq \nu$, mentre per le altre coordinate il gradiente viene reso il più piccolo possibile in valore assoluto.

Mettendo insieme tutti i casi, si ottiene la seguente espressione per il gradiente generalizzato:

$$[\tilde{\nabla} F_S(w)]^i = \begin{cases} \frac{\partial J_S(w)}{\partial w_i} + \nu & \text{se } w_i > 0 \text{ oppure } \left(w_i = 0 \text{ e } \frac{\partial J_S(w)}{\partial w_i} < -\nu \right), \\ \frac{\partial J_S(w)}{\partial w_i} - \nu & \text{se } w_i < 0 \text{ oppure } \left(w_i = 0 \text{ e } \frac{\partial J_S(w)}{\partial w_i} > \nu \right), \\ 0 & \text{se } w_i = 0 \text{ e } -\nu \leq \frac{\partial J_S(w)}{\partial w_i} \leq \nu. \end{cases} \quad (6.3)$$

Nota

La formula (6.3) è la versione multidimensionale del subgradiente di $F_S(w) = J_S(w) + \nu \|w\|_1$. Essa generalizza il caso 1D: quando $w_i \neq 0$, il gradiente è semplicemente la somma del gradiente di J_S e del segno di w_i moltiplicato per ν ; quando $w_i = 0$, si sceglie il subgradiente che punta nella direzione di massima discesa.

2.3.3 Identificazione dell'Active Set e della Faccia Ortante

Poiché la funzione $\nu|w_i|$ ha un angolo (una non-derivabilità) proprio in $w_i = 0$, l'algoritmo non può usare la solita regola della derivata zero per decidere se quel punto è il minimo. Deve quindi fare una scelta esplicita: se la pendenza della parte liscia J è più forte del costo ν , allora conviene 'liberare' w_i (spostarla da zero); altrimenti, conviene 'attivarla' (tenerla bloccata a zero per ottenere una soluzione sparsa).

L'obiettivo della fase di predizione è determinare quali variabili sono nulle alla soluzione e identificare la *faccia ortante* (parte dell'iperpiano) su cui eseguire la minimizzazione. L'idea è quella di compiere un passo infinitesimo lungo la direzione di massima discesa $-\tilde{\nabla} F_{S_k}(w_k)$ e osservare il segno di ogni componente.

Definiamo il vettore $z_k \in \{-1, 0, +1\}^m$ che caratterizza la faccia ortante:

$$z_k^i = \begin{cases} +1 & \text{se } w_k^i > 0 \text{ oppure } \left(w_k^i = 0 \text{ e } \frac{\partial J_{S_k}(w_k)}{\partial w_i} < -\nu \right), \\ -1 & \text{se } w_k^i < 0 \text{ oppure } \left(w_k^i = 0 \text{ e } \frac{\partial J_{S_k}(w_k)}{\partial w_i} > \nu \right), \\ 0 & \text{altrimenti.} \end{cases} \quad (6.4)$$

Il vettore z_k definisce l'ortante

$$\Omega_k = \{d \in \mathbb{R}^m : \text{sign}(d^i) = \text{sign}(z_k^i)\}, \quad (6.5)$$

all'interno del quale la funzione $\|w\|_1$ è differenziabile e il suo gradiente è esattamente z_k . Le variabili con $z_k^i = 0$ costituiscono l'**active set**:

$$\mathcal{A}_k \triangleq \{i \mid z_k^i = 0\}. \quad (6.6)$$

Tali variabili verranno mantenute a zero durante la fase di minimizzazione nel sottospazio; le rimanenti sono dette *libere*.

2.3.4 Fase di Minimizzazione nel Sottospazio

Una volta identificata la faccia ortante, si costruisce un modello quadratico della funzione obiettivo ristretto alle variabili libere. Il modello è:

$$\begin{aligned} \min_{d \in \mathbb{R}^m} m_k(d) &\stackrel{\text{def}}{=} F_{\mathcal{S}_k}(w_k) + \tilde{\nabla} F_{\mathcal{S}_k}(w_k)^T d + \frac{1}{2} d^T \nabla^2 J_{\mathcal{H}_k}(w_k) d \\ \text{s.t. } d^i &= 0, \quad i \in \mathcal{A}_k. \end{aligned} \quad (6.7)$$

La minimizzazione viene effettuata con il metodo del gradiente coniugato Hessian-free applicato al sistema lineare ridotto. Sia Y_k la matrice $n \times (n - |\mathcal{A}_k|)$ le cui colonne sono i versori delle coordinate libere; allora il sistema da risolvere è:

$$[Y_k^T \nabla^2 J_{\mathcal{H}_k}(w_k) Y_k] d^Y = -Y_k^T \tilde{\nabla} F_{\mathcal{S}_k}(w_k), \quad (6.8)$$

dove $d^Y \in \mathbb{R}^{n-|\mathcal{A}_k|}$ è il vettore delle componenti libere. La direzione di ricerca dell'algoritmo è quindi $d_k = Y_k d^Y$.

2.3.5 Ricerca Lineare Proiettata

Per garantire che il nuovo iterato rimanga all'interno della stessa faccia ortante Ω_k , si utilizza una *proiezione ortogonale* $P(\cdot)$ su Ω_k :

$$P(w_i) = \begin{cases} w_i & \text{se } \text{sign}(w_i) = \text{sign}(z_k^i), \\ 0 & \text{altrimenti.} \end{cases} \quad (6.9)$$

La ricerca lineare determina il passo α_k come il più grande valore nella sequenza $1, \frac{1}{2}, \frac{1}{4}, \dots$ tale che:

$$F_{\mathcal{S}_k}(P[w_k + \alpha_k d_k]) \leq F_{\mathcal{S}_k}(w_k) + \sigma \tilde{\nabla} F_{\mathcal{S}_k}(w_k)^T (P[w_k + \alpha_k d_k] - w_k) \quad (6.10)$$

(Armijo) dove $\sigma \in (0, 1)$ è una costante prefissata. Il nuovo iterato è $w_{k+1} = P[w_k + \alpha_k d_k]$.

2.3.6 Algoritmo Completo

Algoritmo

Newton-CG per Problemi L_1 -Regolarizzati

Input: w_0 (punto iniziale, tipicamente $w_0 = 0$), $\mathcal{H}_0, \mathcal{S}_0$ con $|\mathcal{H}_0| < |\mathcal{S}_0|$, $\nu > 0$ (parametro di regolarizzazione), $\eta, \sigma \in (0, 1)$, $\max_{cg}$ (limite iterazioni CG).

Per ogni iterazione $k = 0, 1, \dots$ fino a convergenza:

1. Calcola $\tilde{\nabla} F_{\mathcal{S}_k}(w_k)$ tramite (6.3) e determina z_k con (6.4). Definisci l'active set $\mathcal{A}_k$ con (6.6).
2. Minimizzazione nel sottospazio
 - (a) Calcola $\tilde{\nabla} F_{\mathcal{S}_k}(w_k) = \nabla J_{\mathcal{S}_k}(w_k) + \nu z_k$.
 - (b) Applica il gradiente coniugato al sistema (6.8). Termina quando il residuo r_k soddisfa

$$\|r_k\| \leq \eta \|Y_k^T \tilde{\nabla} F_{\mathcal{S}_k}(w_k)\|, \quad (6.11)$$

oppure dopo $\max_{cg}$ iterazioni.

- (c) Poni $d_k = Y_k d^Y$.

3. Ricerca lineare proiettata. Determina il più grande $\alpha_k \in \{1, \frac{1}{2}, \frac{1}{4}, \dots\}$ che soddisfi (6.10). Aggiorna

$$w_{k+1} = P[w_k + \alpha_k d_k].$$

4. Re-campionamento. Scegli $\mathcal{H}_{k+1}, \mathcal{S}_{k+1}$ con $|\mathcal{H}_{k+1}| = |\mathcal{H}_0|$ e $|\mathcal{S}_{k+1}| = |\mathcal{S}_0|$.

Nota

L'algoritmo appena descritto è particolarmente efficace nella produzione di soluzioni sparse: la fase di predizione identifica rapidamente le variabili nulle, mentre la fase di sottospazio accelera la convergenza sulle variabili libere sfruttando l'informazione di curvatura dell'Hessiana sottocampionata.

3 Visualizzazione Interattiva

Per rendere concreti i concetti teorici esposti nelle sezioni precedenti, è stata realizzata un'applicazione web interattiva che implementa gli algoritmi descritti in un ambiente visivo. L'applicazione è accessibile tramite browser e consente di sperimentare il comportamento dei metodi a campionamento dinamico su funzioni obiettivo modificabili dall'utente.

Il codice Python (funzione obiettivo, gradiente, algoritmo) viene eseguito nel browser tramite un runtime Python, consentendo all'utente di modificare loss e

algoritmo e vedere il risultato. La visualizzazione delle superfici e dei percorsi è affidata a Plotly (libreria JavaScript).

L'interfaccia permette di selezionare o scrivere una funzione obiettivo $J(w)$ con gradiente $\nabla J(w)$ e Hessiana $\nabla^2 J(w)$, scegliendo tra preset predefiniti o codice personalizzato; di scegliere tra i tre algoritmi discussi (gradiente dinamico, Newton-CG con campionamento dinamico, Newton-CG con regolarizzazione L_1), il cui codice Python è visibile e modificabile; di variare i parametri dell'algoritmo in tempo reale; di visualizzare il percorso di ottimizzazione sulla superficie di $J(w)$ (in 3D per funzioni 2D, in 2D per funzioni 1D) con possibilità di scorrere le iterazioni, avviare una riproduzione automatica e ruotare la visualizzazione; di monitorare metriche in tempo reale; e di eseguire un'analisi di convergenza che mostra una tabella dell'errore e di $J(w_k)$ per le iterazioni. *Nel codice HTML, la superficie plottata è la funzione obiettivo teorica $J(w)$, definita dall'utente nel preset della loss. Questa è la funzione che si vorrebbe minimizzare e viene usata per disegnare il grafico, calcolare i valori del percorso e mostrare le metriche come $J(w_k)$ e $\|\nabla J(w_k)\|$. L'algoritmo non minimizza direttamente $J(w)$, ma utilizza un dataset sintetico di N funzioni di perdita individuali $\ell(w; i)$, con $i = 1, \dots, N$, costruito a partire da $J(w)$ perturbando i suoi coefficienti. Ogni funzione $\ell(w; i)$ rappresenta il contributo del singolo esempio i . A ogni iterazione, l'algoritmo seleziona un batch $\mathcal{S}_k$ di dimensione n_k e calcola il gradiente del batch come $g_k = \frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \nabla \ell(w_k; i)$. La line search di Armijo usa la loss del batch $J_{\mathcal{S}_k}(w) = \frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \ell(w; i)$. L'aggiornamento dei pesi $w_{k+1} = w_k - \alpha_k g_k$ minimizza quindi la media campionaria sul batch corrente. La media su tutti gli N dati è $\hat{J}_N(w) = \frac{1}{N} \sum_{i=1}^N \ell(w; i)$. Il dataset è costruito in modo che le medie campionarie dei coefficienti delle perturbazioni coincidano esattamente con i coefficienti di $J(w)$; pertanto $\hat{J}_N(w) = J(w)$ per ogni w e $\nabla \hat{J}_N(w) = \nabla J(w)$ per ogni w , indipendentemente da N . Questa proprietà è ottenuta sottraendo a ogni perturbazione la propria media campionaria. Quindi $J(w)$ è la funzione plottata, mentre l'algoritmo minimizza $\hat{J}_N(w)$ (o sue approssimazioni su batch). Nel codice $\hat{J}_N(w)$ coincide esattamente con $J(w)$ per ogni N , a differenza di un problema reale dove la coincidenza si ha solo nel limite $N \rightarrow \infty$. In sintesi, la loss teorica $J(w)$ è usata per il plot e le metriche, la loss campionaria totale $\hat{J}_N(w)$ per la condizione di arresto, la loss del batch $J_{\mathcal{S}_k}(w)$ per la line search, e il gradiente del batch g_k per l'aggiornamento dei pesi. La loss del batch e il gradiente del batch sono approssimazioni stocastiche che coincidono con le quantità esatte solo nel limite $n_k \rightarrow N$ e $N \rightarrow \infty$.*

L'implementazione segue la struttura algoritmica descritta nel documento, con due estensioni rispetto alla formulazione originale: la CCV estesa al subgradiente per L_1 (nel paper il sample size per il caso L_1 è assunto fisso, e gli autori lasciano l'estensione al caso dinamico come lavoro futuro) e il dataset sintetico centrato (costruito in modo che $\hat{J}_N(w) = J(w)$ esattamente per ogni N , una semplificazione che nei problemi reali si avrebbe solo nel limite $N \rightarrow \infty$). Per il resto, l'implementazione è fedele al paper: l'Hessiana nel Newton-CG è calcolata su un sottocampione di dimensione $|\mathcal{H}_k| = R \cdot |\mathcal{S}_k|$; il CG per il caso L_1 usa la tolleranza fissa η dell'Equation (6.11); la line search di Wolfe è quella degli algoritmi pratici del paper, mentre i passi fissi compaiono solo nell'analisi teorica di complessità.

3.1 Metodo del Gradiente a Campione Dinamico

Dynamic GD

```
import numpy as np

def dynamic_gd(w0, theta, max_iter, alpha, batch0):
    w = np.array(w0, dtype=float)
    n = max(batch0, 2)
    history = [w.copy().tolist()]
    batch_sizes = [n]
```

Definisce una funzione che prende in input w_0 , θ , $\max_iter$, α , batch_0 , poi inizializza il vettore dei pesi w_0 (dove $w_0 \in \mathbb{R}^d$, con d pari al numero di parametri del modello), imposta la dimensione del batch iniziale $n = n_0 = \max(\text{batch}_0, 2)$ in modo che non possa essere minore di 2, e inizializza la lista ‘history’ (che conterrà i pesi w_k a ogni iterazione k) e ‘batch_sizes’ (che conterrà le corrispondenti dimensioni n_k di ogni batch).

Dynamic GD

```
#
for k in range(max_iter):
    indices = np.random.choice(N, size=n, replace=False)
```

Per ogni iterazione $k \in \{0, \dots, \max_iter\}$ crea una lista indices ovvero un campione casuale $\mathcal{S}_k = \{i_1, \dots, i_n\}$ di n indici distinti da $\{1, \dots, N\}$, estratto senza reinserimento (ovvero uniforme tra tutti i $\binom{N}{n}$ possibili sottoinsiemi). N è definito come il numero di esempi totali, nel codice Python, per questioni di indicizzazione, l'insieme è $\{0, \dots, N-1\}$.

Dynamic GD

```
#
grads = np.array([grad_i(w, i) for i in indices])
g = np.mean(grads, axis=0)
```

grads colleziona un insieme di gradienti di loss su tanti esempi diversi (uno per ogni indice) quindi g è l'approssimazione che viene fatta del gradiente della loss usando il batch. Matematicamente grads è una lista di $\nabla \ell(w_k; i)$ per ogni indice i con cui si costruisce il batch mentre $g = g_k = \nabla J_{\mathcal{S}_k}(w_k) = \frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \nabla \ell(w_k; i)$, dove $n_k = |\mathcal{S}_k|$ è la dimensione del batch corrente.

Dynamic GD

```
#
if n > 1:
    var_vec = np.var(grads, axis=0, ddof=1)
else:
    var_vec = np.zeros_like(g)
```

```

V_norm1 = np.sum(var_vec)

gg = np.dot(g, g)
if gg > 1e-16:
    if V_norm1 / n > theta**2 * gg:
        n_new = int(np.ceil(V_norm1 / (theta**2 * gg))) + 1
        n = min(n_new, N)

```

Matematicamente, `var_vec` è un vettore che calcola la varianza campionaria di ogni componente del gradiente sui dati del batch: $\hat{\mathcal{V}} := \text{Var}_{i \in \mathcal{S}_k}(\nabla \ell(w_k; i)) = \frac{1}{n_k - 1} \sum_{i \in \mathcal{S}_k} (\nabla \ell(w_k; i) - g_k)^2 \in \mathbb{R}^d$, dove $g_k = \nabla J_{\mathcal{S}_k}(w_k)$ come già definito. Poi `V_norm1` ne prende la norma L_1 : $\|\hat{\mathcal{V}}\|_1 = \sum_{j=1}^d [\hat{\mathcal{V}}]_j$. Questa quantità viene confrontata con $\theta^2 \|g_k\|_2^2$ per decidere se aumentare il batch, quindi se la CCV è soddisfatta allora la dimensione del batch viene aumentata come descritto dalla regola di aggiornamento del batch. Se $n = 1$ (mai vero, il controllo è superfluo) la varianza campionaria non è definita (divisione per 0), quindi si pone $\hat{\mathcal{V}} = 0$

Dynamic GD

```

#
def J_batch(w_curr):
    return np.mean([loss_i(w_curr, i) for i in indices])

c1 = 1e-4
step = alpha
J_curr = J_batch(w)
g_norm2 = np.dot(g, g)

```

Definisce la funzione di perdita sul batch corrente $J_batch = J_{\mathcal{S}_k}(w) = \frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \ell(w; i)$, definisce lo step che viene inizializzato come `alpha`, definisce la variabile `J_curr` chiamando la funzione `J_batch` sui parametri `w` e definisce `g_norm2` come $g_k^T g_k = \|g_k\|_2^2$

Dynamic GD

```

#
def J_batch(w_curr):
    return np.mean([loss_i(w_curr, i) for i in indices])

# Line search Wolfe
c1, c2 = 1e-4, 0.9
step = alpha
J_curr = J_batch(w)
g_norm2 = np.dot(g, g)
d = -g
gd = -g_norm2
if g_norm2 > 1e-16:
    for _ in range(30):
        w_new = w + step * d

```

```

        if J_batch(w_new) <= J_curr + c1 * step * gd:
            g_new = np.mean([grad_i(w_new, i) for i in
                             indices], axis=0)
            if np.dot(g_new, d) >= c2 * gd:
                break
            step *= 0.5
        else:
            step = 0.0

    w = w + step * d

```

Si definisce la funzione di perdita sul batch corrente $J_{\mathcal{S}_k}(w)$. La direzione di ricerca è fissata come $d_k = -g_k$ e il prodotto scalare $g_k^\top d_k = -\|g_k\|_2^2$ misura la pendenza iniziale lungo tale direzione. Se il gradiente non è trascurabile ($\|g_k\|_2^2 > 10^{-16}$), si esegue una line search con backtracking che richiede le condizioni di Wolfe. $J_{\mathcal{S}_k}(w_k + \alpha_k d_k) \leq J_{\mathcal{S}_k}(w_k) + c_1 \alpha_k g_k^\top d_k$ con $c_1 = 10^{-4}$. Se questa è soddisfatta, si calcola il gradiente nel punto di prova sullo stesso batch e si verifica la seconda condizione (curvatura): il gradiente nel nuovo punto deve essere meno negativo lungo d_k , ovvero $\nabla J_{\mathcal{S}_k}(w_k + \alpha_k d_k)^\top d_k \geq c_2 g_k^\top d_k$ con $c_2 = 0.9$. Se una delle due condizioni fallisce, il passo viene dimezzato. Dopo al più 30 tentativi, se nessuno step è accettabile si pone $\alpha_k = 0$. Infine si aggiorna $w_{k+1} = w_k + \alpha_k d_k$.

Dynamic GD

```

if np.linalg.norm(grad_full(w)) < 1e-6:
    history.append(w.copy().tolist())
    batch_sizes.append(n)
    break

history.append(w.copy().tolist())
batch_sizes.append(n)

return history, batch_sizes

history, batch_sizes = dynamic_gd(w0, theta, max_iter, alpha, batch0)

```

Infine, si controlla la norma del gradiente esatto (calcolato su tutti i dati) tramite $\text{grad_full}(w)$: se $\|\nabla J(w_k)\|_2 < 10^{-6}$, si considera raggiunta la convergenza e si interrompe il ciclo, registrando l'ultimo punto. Altrimenti, si continua registrando la storia dei pesi e delle dimensioni del batch. Al termine, la funzione restituisce le liste `history` (contenente i pesi w_k a ogni iterazione) e `batch_sizes` (contenente le corrispondenti dimensioni n_k del batch).

3.2 Metodo di Newton-CG con Campionamento Dinamico

Newton-CG

```
import numpy as np

def cg(A, b, gamma, maxcg):
    x = np.zeros_like(b)
    r = b - A(x)
    p = r.copy()
    rr = np.dot(r, r)
    for _ in range(maxcg):
        Ap = A(p)
        pHp = np.dot(p, Ap)
        if pHp <= 1e-14:
            break
        alpha = rr / pHp
        x = x + alpha * p
        r_new = r - alpha * Ap
        rr_new = np.dot(r_new, r_new)
        if rr_new <= gamma * np.dot(x, x) + 1e-16:
            return x
        beta = rr_new / rr
        p = r_new + beta * p
        r = r_new
        rr = rr_new
    return x
```

La funzione `cg` risolve $Ax = b$ con il metodo del gradiente coniugato, dove A è l'operatore Hessiana-vettore. Parte da $x_0 = 0$, residuo $r_0 = b$ e direzione $p_0 = r_0$. Ad ogni iterazione calcola Ap , la curvatura $pHp = p^T Ap$ (se troppo piccola, si ferma), aggiorna con $\alpha = rr/pHp$ e il residuo. Il test di arresto

$$\|r_{\text{new}}\|^2 \leq \gamma \|x\|^2 + 10^{-16}$$

è descritto nel paragrafo “Criterio di terminazione del gradiente coniugato” la soglia γ (calcolata dall'algoritmo esterno) bilancia l'errore di troncamento del CG con il rumore statistico del sottocampionamento dell'Hessiana. Se il test è soddisfatto, restituisce x ; altrimenti aggiorna $\beta = rr_{\text{new}}/rr$ e la direzione $p = r_{\text{new}} + \beta p$. Il ciclo termina al massimo dopo `maxcg` iterazioni.

Newton-CG

```
def newton_cg(w0, theta, max_iter, alpha, batch0, R, maxcg):
    w = np.array(w0, dtype=float)
    n = max(batch0, 2)
    history, batch_sizes = [w.copy().tolist()], [n]
```

La funzione principale `newton_cg` riceve il punto iniziale w_0 , la tolleranza θ per il controllo della varianza, il numero massimo di iterazioni esterne `max_iter`, il passo iniziale `alpha` per la line search, la dimensione del batch iniziale `batch0`, il rapporto

R tra la dimensione del campione per l'Hessiana e quello per il gradiente, e il numero massimo di iterazioni del CG `maxcg`. Il vettore dei pesi w viene convertito in array NumPy in doppia precisione, la dimensione del batch è forzata ad almeno 2 (per garantire che la varianza campionaria sia definita) e si inizializzano le liste `history` e `batch_sizes` che registreranno la traiettoria dei pesi e le dimensioni del batch a ogni iterazione.

Newton-CG

```
#
for k in range(max_iter):
    indices_S = np.random.choice(N, size=n, replace=False)
    g = np.mean([grad_i(w, i) for i in indices_S], axis=0)
```

Ad ogni iterazione esterna k , si estrae un campione casuale $\mathcal{S}_k$ di $n_k = n$ indici distinti (senza reinserimento) da $\{0, \dots, N - 1\}$. Su questo campione si calcola il gradiente approssimato

$$g_k = \nabla J_{\mathcal{S}_k}(w_k) = \frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \nabla \ell(w_k; i),$$

che costituisce la stima stocastica del gradiente esatto $\nabla J(w_k)$.

Newton-CG

```
#
n_h = min(max(1, int(round(R * n))), N)
indices_H = np.random.choice(indices_S, size=n_h,
    replace=False)
Hv = lambda v: np.mean([hessvec_i(w, i, v) for i in
    indices_H], axis=0)
```

Si determina la dimensione del campione per l'Hessiana come $n_h = \lceil R n_k \rceil$, vincolata tra 1 e N . Viene quindi estratto un sottocampione $\mathcal{H}_k \subseteq \mathcal{S}_k$ (senza reinserimento) di dimensione n_h . L'operatore `Hv` definisce il prodotto Hessiana-vettore come

$$H_v(v) = \nabla^2 J_{\mathcal{H}_k}(w_k) v = \frac{1}{n_h} \sum_{i \in \mathcal{H}_k} \nabla^2 \ell(w_k; i) v.$$

Questo operatore viene passato al risolutore CG, che lo utilizzerà per calcolare i prodotti Ap senza mai costruire esplicitamente la matrice Hessiana.

Newton-CG

```
#
p0 = -g
p0_norm2 = np.dot(p0, p0)
gamma = 0.0
if p0_norm2 > 1e-16 and n_h > 1:
    Hp0 = np.array([hessvec_i(w, i, p0) for i in indices_H])
    gamma = np.sum(np.var(Hp0, axis=0, ddof=1)) / (n_h *
        p0_norm2)
```

Prima di chiamare il CG, si calcola il coefficiente γ che quantifica l'errore di approssimazione dell'Hessiana lungo la direzione iniziale $p_0 = -g_k$. Si valutano i prodotti Hessiana-vettore individuali $\nabla^2 \ell(w_k; i) p_0$ per ogni $i \in \mathcal{H}_k$; la loro varianza campionaria (componente per componente) viene aggregata in norma L_1 e normalizzata:

$$\gamma = \frac{\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) p_0)\|_1}{n_h \|p_0\|_2^2}.$$

Se il gradiente è nullo (o quasi) o $n_h = 1$ (varianza non definita), si pone $\gamma = 0$, il che corrisponde a richiedere convergenza completa del CG. Questo parametro sarà utilizzato nel test di arresto del CG per fermarsi quando il residuo è confrontabile con il rumore statistico dell'Hessiana.

Newton-CG

```
#
d = cg(Hv, -g, gamma, maxcg)
```

Si risolve il sistema lineare di Newton approssimato

$$\nabla^2 J_{\mathcal{H}_k}(w_k) d = -g_k$$

chiamando la funzione `cg` con l'operatore `Hv`, il termine noto $-g_k$, il coefficiente γ e il limite di iterazioni `maxcg`. La soluzione d è la direzione di ricerca di Newton-CG, calcolata senza costruire l'Hessiana e con un criterio di arresto che bilancia l'accuratezza interna con l'incertezza statistica del campionamento.

Newton-CG

```
#
J_batch = lambda wc: np.mean([loss_i(wc, i) for i in
    indices_S])

c1, c2 = 1e-4, 0.9
step, J_w = alpha, J_batch(w)
gd = np.dot(g, d)
if gd >= 0:
    d = -g
    gd = -np.dot(g, g)
```

Si definisce la funzione obiettivo sul batch del gradiente, $J_{\mathcal{S}_k}(w)$, e si inizializza la ricerca lineare con passo `step = alpha`. Si calcola la derivata direzionale $\text{gd} = g_k^T d$. Se $\text{gd} \geq 0$, la direzione d non è di discesa; in tal caso si forza il fallback $d = -g_k$, che è sempre di discesa (a meno di gradiente nullo), e si ricalcola $\text{gd} = -\|g_k\|^2$.

Newton-CG

```
#
for _ in range(30):
    w_new = w + step * d
    if J_batch(w_new) <= J_w + c1 * step * gd:
```



```

g_new = np.mean([grad_i(w_new, i) for i in
                 indices_S], axis=0)
if np.dot(g_new, d) >= c2 * gd:
    break
step *= 0.5
else:
    step = 0.0
w = w + step * d

```

La line search con backtracking cerca un passo α_k che soddisfi le condizioni di Wolfe. $J_{S_k}(w_k + \alpha_k d_k) \leq J_{S_k}(w_k) + c_1 \alpha_k g_k^T d_k$ con $c_1 = 10^{-4}$ e $\nabla J_{S_k}(w_k + \alpha_k d_k)^T d_k \geq c_2 g_k^T d_k$ con $c_2 = 0.9$ (Spiegazione nell'appendice). Se entrambe sono soddisfatte, il passo è accettato. In caso contrario, il passo viene dimezzato e si riprova, per al più 30 tentativi; se nessuno è accettabile, si pone $\alpha_k = 0$. L'aggiornamento dei pesi è quindi $w_{k+1} = w_k + \alpha_k d_k$.

Newton-CG

```

#
history.append(w.copy().tolist())
batch_sizes.append(n)

```

Si registrano il nuovo punto w_{k+1} e la dimensione corrente del batch n .

Newton-CG

```

#
indices_new = np.random.choice(N, size=n, replace=False)
g_new = np.mean([grad_i(w, i) for i in indices_new], axis=0)
var_vec = np.var([grad_i(w, i) for i in indices_new], axis=0,
                 ddof=1) if n > 1 else np.zeros_like(g_new)
V_norm1, gg_new = np.sum(var_vec), np.dot(g_new, g_new)
if gg_new > 1e-16 and V_norm1 / n > theta**2 * gg_new:
    n = min(int(np.ceil(V_norm1 / (theta**2 * gg_new))) + 1,
            N)

```

Dopo l'aggiornamento dei pesi, si verifica la Condizione di Controllo della Varianza (CCV) su un nuovo campione $\mathcal{S}_{k+1}$ di dimensione n (quella corrente) per decidere se aumentare il batch per l'iterazione successiva. Si calcolano i gradienti individuali sul nuovo campione, la loro varianza campionaria $\hat{\mathcal{V}}$ e la norma L_1 $V_{\text{norm1}} = \|\hat{\mathcal{V}}\|_1$. La CCV richiede

$$\frac{\|\hat{\mathcal{V}}\|_1}{n} \leq \theta^2 \|g_{\text{new}}\|_2^2.$$

Se non è soddisfatta, la nuova dimensione del batch viene stimata come

$$n \leftarrow \min \left\{ N, \left\lceil \frac{\|\hat{\mathcal{V}}\|_1}{\theta^2 \|g_{\text{new}}\|_2^2} \right\rceil + 1 \right\},$$

in modo che il batch cresca automaticamente quando la varianza dello stimatore è troppo alta rispetto alla norma del gradiente, ovvero quando ci si avvicina alla soluzione e serve maggiore precisione.

Newton-CG

```
#
    if np.linalg.norm(grad_full(w)) < 1e-6:
        break
    return history, batch_sizes

history, batch_sizes = newton_cg(w0, theta, max_iter, alpha, batch0,
                                R, maxcg)
```

Infine, si controlla la norma del gradiente esatto (calcolato su tutto il dataset) tramite `grad_full`; se $\|\nabla J(w_{k+1})\|_2 < 10^{-6}$, si considera raggiunta la convergenza e si interrompe il ciclo. Al termine delle iterazioni, la funzione restituisce le liste `history` e `batch_sizes`, che contengono rispettivamente la traiettoria dei pesi w_k e le dimensioni del batch n_k a ogni iterazione. L'ultima riga mostra la chiamata tipica alla funzione, che produce le due liste per l'analisi successiva. L'algoritmo Newton CG è sostanzialmente Dynamic GD ma la direzione di discesa è quella di Newton anziché l'antigradiente

3.3 Metodo Newton-CG per Modelli con Regolarizzazione L_1

Newton-L1

```
import numpy as np

def newton_l1(w0, theta, max_iter, alpha, batch0, nu, sigma, maxcg,
             eta=0.5):
    w = np.array(w0, dtype=float)
    n = max(batch0, 2)
    history = [w.copy().tolist()]
    batch_sizes = [n]
```

La funzione principale `newton_l1` implementa il metodo Newton-CG con campionamento dinamico per problemi con regolarizzazione L_1 . Riceve il punto iniziale w_0 , la tolleranza θ per il controllo della varianza, il numero massimo di iterazioni esterne `max_iter`, il passo iniziale `alpha` per la ricerca lineare, la dimensione del batch iniziale `batch0`, il parametro di penalità $\nu > 0$, la costante $\sigma \in (0, 1)$ per la condizione di Armijo, il numero massimo di iterazioni del gradiente coniugato `maxcg` e il fattore di tolleranza $\eta \in (0, 1)$ per il criterio di arresto del CG. Il vettore dei pesi w viene convertito in array NumPy in doppia precisione, la dimensione del batch è forzata ad almeno 2 (per garantire che la varianza campionaria sia definita) e si inizializzano le liste `history` e `batch_sizes` che registreranno la traiettoria dei pesi e le dimensioni del batch a ogni iterazione.

Newton-L1

```
#
def F_batch(v, indices):
    Jb = np.mean([loss_i(v, i) for i in indices])
    return Jb + nu * np.sum(np.abs(v))
def subgrad_batch(v, indices):
    grads = np.array([grad_i(v, i) for i in indices])
    gJ = np.mean(grads, axis=0)
    g = np.zeros_like(v)
    for i in range(len(v)):
        if v[i] > 0:
            g[i] = gJ[i] + nu
        elif v[i] < 0:
            g[i] = gJ[i] - nu
        else:
            if gJ[i] < -nu:
                g[i] = gJ[i] + nu
            elif gJ[i] > nu:
                g[i] = gJ[i] - nu
            else:
                g[i] = 0.0
    return g
def project_orthant(v, z):
    res = v.copy()
```

```

for i in range(len(v)):
    if z[i] != 0 and np.sign(res[i]) != z[i]:
        res[i] = 0.0
return res

```

Vengono definite tre funzioni ausiliarie interne. `F_batch` calcola la funzione obiettivo regolarizzata sul batch $\mathcal{S}$:

$$F_{\mathcal{S}}(v) = J_{\mathcal{S}}(v) + \nu \|v\|_1 = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \ell(v; i) + \nu \sum_{j=1}^m |v_j|,$$

in accordo con la formulazione (6.2). `subgrad_batch` implementa il gradiente generalizzato $\tilde{\nabla} F_{\mathcal{S}}(v)$ componente per componente, seguendo la regola (6.3): se $v_i \neq 0$ si somma $\nu \operatorname{sign}(v_i)$; se invece $v_i = 0$, si sceglie il subgradiente in $[-\nu, +\nu]$ che rende nullo (quando possibile) la componente del gradiente generalizzato, realizzando la proiezione $g_i = \arg \min_{g \in [-\nu, \nu]} |(\nabla J_{\mathcal{S}})_i + g|$. `project_orthant` implementa la proiezione $P(\cdot)$ sulla faccia ortante Ω_k definita in (6.5): azzera ogni componente il cui segno non coincide con quello prescritto dal vettore z .

Newton-L1

```

#
for k in range(max_iter):
    indices_S = np.random.choice(N, size=n, replace=False)
    grads = np.array([grad_i(w, i) for i in indices_S])
    g_batch = np.mean(grads, axis=0)
    z = np.where(w > 0, 1,
                np.where(w < 0, -1,
                        np.where(g_batch < -nu, 1,
                                np.where(g_batch > nu, -1, 0))))

```

Ad ogni iterazione esterna k , si estrae un campione casuale $\mathcal{S}_k$ di $n_k = n$ indici distinti (senza reinserimento) da $\{0, \dots, N-1\}$. Su questo campione si calcola il gradiente della parte liscia

$$g_k = \nabla J_{\mathcal{S}_k}(w_k) = \frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \nabla \ell(w_k; i),$$

che costituisce la stima stocastica del gradiente esatto $\nabla J(w_k)$. Quindi si determina il vettore $z_k \in \{-1, 0, +1\}^m$ che caratterizza la faccia ortante secondo la (6.4): $z_k^i = \operatorname{sign}(w_k^i)$ se $w_k^i \neq 0$; se invece $w_k^i = 0$, la componente viene posta a ± 1 quando il gradiente della parte liscia supera in valore assoluto la soglia ν (indicando che conviene “liberare” la variabile), e a 0 altrimenti (la variabile resta bloccata nell’active set).

Newton-L1

```

#
sg = subgrad_batch(w, indices_S)
sgn = np.linalg.norm(sg)

```

```

if sgn < 1e-10:
    history.append(w.copy().tolist())
    batch_sizes.append(n)
    break
n_h = max(1, int(round(R * n)))
n_h = min(n_h, N)
indices_H = np.random.choice(indices_S, size=n_h,
    replace=False)
free = (z != 0)
d = np.zeros_like(w)
if np.any(free):
    g_free = sg[free]
    # Hessian-free: prodotto H·v senza costruire H
    def Hv(v_full):
        return np.mean([hessvec_i(w, i, v_full) for i in
            indices_H], axis=0)
    def Hv_free(v_free):
        v_full = np.zeros_like(w)
        v_full[free] = v_free
        Hv_full = Hv(v_full)
        return Hv_full[free]
    tol_cg = eta * np.linalg.norm(g_free)
    d_free = np.zeros(np.sum(free))
    r = -g_free.copy()
    p = r.copy()
    rr = np.dot(r, r)
    for _ in range(maxcg):
        Hp = Hv_free(p)
        pHp = np.dot(p, Hp)
        if pHp <= 1e-14:
            if np.linalg.norm(d_free) < 1e-14:
                d_free = -g_free.copy()
            break
        alpha_cg = rr / pHp
        d_free = d_free + alpha_cg * p
        r_new = r - alpha_cg * Hp
        rr_new = np.dot(r_new, r_new)
        if np.sqrt(rr_new) <= tol_cg:
            r = r_new
            rr = rr_new
            break
        beta = rr_new / rr
        p = r_new + beta * p
        r = r_new
        rr = rr_new
    d[free] = d_free

```

Si calcola il subgradiente $\tilde{\nabla} F_{S_k}(w_k)$ tramite `subgrad_batch` e la sua norma euclidea; se questa è trascurabile ($< 10^{-10}$) si considera raggiunta la convergenza e si interrompe il ciclo, registrando l'ultimo punto. Altrimenti si determina la dimensione del campione per l'Hessiana come $n_h = \lceil R n_k \rceil$, vincolata tra 1 e N , e si estrae il

sottocampione $\mathcal{H}_k \subseteq \mathcal{S}_k$ (senza reinserimento) di dimensione n_h . Il vettore booleano **free** individua le variabili libere (quelle con $z_k^i \neq 0$), mentre le rimanenti appartengono all'active set $\mathcal{A}_k$ definito in (6.6). Sul sottospazio delle variabili libere si risolve il sistema lineare di Newton ridotto

$$[Y_k^T \nabla^2 J_{\mathcal{H}_k}(w_k) Y_k] d^Y = -Y_k^T \tilde{\nabla} F_{\mathcal{S}_k}(w_k),$$

dove Y_k è la matrice dei versori delle coordinate libere, mediante il metodo del gradiente coniugato *Hessian-free*. Gli operatori **Hv** e **Hv_free** calcolano il prodotto $\nabla^2 J_{\mathcal{H}_k}(w_k) v$ senza costruire esplicitamente la matrice Hessiana. Il CG viene arrestato quando il residuo soddisfa il criterio (6.11),

$$\|r_j\|_2 \leq \eta \|g_{\text{free}}\|_2.$$

o quando incontra curvatura non positiva ($\text{pHp} \leq 10^{-14}$), nel qual caso si effettua il fallback sulla direzione dell'antigradiente ridotto $-g_{\text{free}}$.

Newton-L1

```
#
step = alpha
F_w = F_batch(w, indices_S)
sg_d = np.dot(sg, d)
if sg_d >= 0:
    d = -sg
    sg_d = -np.dot(sg, sg)
w_new = w.copy()
for _ in range(20):
    w_trial = project_orthant(w + step * d, z)
    if F_batch(w_trial, indices_S) <= F_w + sigma * step *
        sg_d:
        w_new = w_trial
        break
    step *= 0.5
    if step < 1e-12:
        w_new = w.copy()
        break
w = w_new
```

Si inizializza la ricerca lineare proiettata con passo **step** = **alpha**. Si calcola il valore dell'obiettivo $F_{\mathcal{S}_k}(w_k)$ e la derivata direzionale $\text{sg_d} = \tilde{\nabla} F_{\mathcal{S}_k}(w_k)^T d$. Se questa non è negativa, la direzione d non è di discesa e si forza il fallback $d = -\tilde{\nabla} F_{\mathcal{S}_k}(w_k)$, che è sempre di discesa (a meno di subgradiente nullo). La line search cerca il più grande passo α_k nella sequenza $1, \frac{1}{2}, \frac{1}{4}, \dots$ tale che

$$F_{\mathcal{S}_k}(P[w_k + \alpha_k d]) \leq F_{\mathcal{S}_k}(w_k) + \sigma \alpha_k \tilde{\nabla} F_{\mathcal{S}_k}(w_k)^T (P[w_k + \alpha_k d] - w_k),$$

dove P è la proiezione **project_orthant**. Questo corrisponde alla condizione (6.10). Se lo step diventa troppo piccolo ($< 10^{-12}$) si rifiuta l'aggiornamento e si mantiene il punto corrente.

Newton-L1

```

#
indices_new = np.random.choice(N, size=n, replace=False)
grads_new = np.array([grad_i(w, i) for i in indices_new])
g_new = np.mean(grads_new, axis=0)
if n > 1:
    var_vec = np.var(grads_new, axis=0, ddof=1)
else:
    var_vec = np.zeros_like(g_new)
V_norm1 = np.sum(var_vec)
gg_new = np.dot(g_new, g_new)
if gg_new > 1e-16:
    if V_norm1 / n > theta**2 * gg_new:
        n_new = int(np.ceil(V_norm1 / (theta**2 * gg_new))) + 1
        n = min(n_new, N)
history.append(w.copy().tolist())
batch_sizes.append(n)
if np.linalg.norm(grad_full(w)) < 1e-6:
    break
return history, batch_sizes

history, batch_sizes = newton_l1(w0, theta, max_iter, alpha, batch0,
    nu, sigma, maxcg, eta)

```

Dopo l'aggiornamento dei pesi, si verifica la Condizione di Controllo della Varianza (CCV) su un nuovo campione $\mathcal{S}_{k+1}$ della stessa dimensione n_k corrente per decidere se aumentare il batch per l'iterazione successiva. Si calcolano i gradienti individuali della parte liscia sul nuovo campione, la loro varianza campionaria $\hat{\mathcal{V}}$ e la norma L_1 $V_{\text{norm1}} = \|\hat{\mathcal{V}}\|_1$. La CCV richiede

$$\frac{\|\hat{\mathcal{V}}\|_1}{n} \leq \theta^2 \|g_{k+1}\|_2^2,$$

dove g_{k+1} è il gradiente della parte liscia sul nuovo campione. Se la condizione non è soddisfatta, la nuova dimensione del batch viene stimata come

$$n \leftarrow \min \left\{ N, \left\lceil \frac{\|\hat{\mathcal{V}}\|_1}{\theta^2 \|g_{k+1}\|_2^2} \right\rceil + 1 \right\}.$$

Questa è un'estensione dinamica rispetto al paper originale, dove il campione per il caso L_1 era mantenuto fisso. Si registrano il nuovo punto w_{k+1} e la dimensione del batch, quindi si controlla la norma del gradiente esatto (calcolato su tutto il dataset) tramite `grad_full`; se $\|\nabla J(w_{k+1})\|_2 < 10^{-6}$ si interrompe il ciclo. Al termine delle iterazioni, la funzione restituisce le liste `history` e `batch_sizes`. L'ultima riga mostra la chiamata tipica alla funzione, che produce le due liste per l'analisi successiva.

4 Appendice

4.1 Dimostrazione delle disuguaglianze di Polyak–Lojasiewicz

Sia w_* l'unico minimo di J e, per semplificare le notazioni, poniamo $J(w_*) = 0$. Allora con le ipotesi strutturali esplicitate sopra valgono le seguenti disuguaglianze:

$$\|\nabla J(w)\|_2^2 \geq 2\lambda J(w), \quad J(w) \geq \frac{\lambda}{2L^2} \|\nabla J(w)\|_2^2.$$

Come prima cosa cerchiamo una rappresentazione integrale esatta di $J(w)$: per ottenere una formula che coinvolga esplicitamente l'Hessiana, applichiamo il Teorema Fondamentale del Calcolo due volte.

Prima applicazione: definiamo $\varphi(t) = J(w_* + t(w - w_*))$ per $t \in [0, 1]$. Allora $\varphi(0) = J(w_*) = 0$, $\varphi(1) = J(w)$, e $\varphi'(t) = \nabla J(w_* + t(w - w_*))^T(w - w_*)$. Per il TFC:

$$J(w) = \underbrace{J(w_*)}_{\varphi(0)=0} + \int_0^1 \nabla J(w_* + t(w - w_*))^T(w - w_*) dt. \quad (\text{A.1})$$

Seconda applicazione: applichiamo ora il TFC alla funzione $\psi(t) = \nabla J(w_* + t(w - w_*))$. Si ha $\psi(0) = \nabla J(w_*) = 0$, $\psi(1) = \nabla J(w)$, e $\psi'(t) = \nabla^2 J(w_* + t(w - w_*))(w - w_*)$. Quindi per ogni $t \in [0, 1]$:

$$\nabla J(w_* + t(w - w_*)) = \int_0^t \nabla^2 J(w_* + s(w - w_*))(w - w_*) ds. \quad (\text{A.2})$$

Sostituiamo (A.2) nella (A.1):

$$J(w) = \int_0^1 \left(\int_0^t \nabla^2 J(w_* + s(w - w_*))(w - w_*) ds \right)^T (w - w_*) dt.$$

Poiché $(w - w_*)$ è costante rispetto agli integrali, possiamo riscrivere:

$$J(w) = \int_0^1 \int_0^t (w - w_*)^T \nabla^2 J(w_* + s(w - w_*))(w - w_*) ds dt.$$

La regione di integrazione è $0 \leq s \leq t \leq 1$, che è equivalente a $0 \leq s \leq 1$, $s \leq t \leq 1$. Scambiando l'ordine di integrazione:

$$J(w) = \int_0^1 \int_s^1 (w - w_*)^T \nabla^2 J(w_* + s(w - w_*))(w - w_*) dt ds.$$

L'integrale interno in dt vale $\int_s^1 dt = 1 - s$. Quindi otteniamo la formula finale:

$$J(w) = \int_0^1 (1 - t)(w - w_*)^T \nabla^2 J(w_* + t(w - w_*))(w - w_*) dt. \quad (\text{A.3})$$

Si consideri ora la derivazione della disuguaglianza $\|\nabla J(w)\|^2 \geq 2\lambda J(w)$. Scriviamo la rappresentazione integrale di $J(y)$ a partire da $J(w)$:

$$J(y) = J(w) + \int_0^1 \nabla J(w + t(y - w))^T(y - w) dt. \quad (\text{A.4})$$

Applicando nuovamente il TFC al gradiente:

$$\nabla J(w + t(y - w)) = \nabla J(w) + \int_0^t \nabla^2 J(w + s(y - w))(y - w) ds. \quad (\text{A.5})$$

Sostituendo (A.5) in (A.4):

$$J(y) = J(w) + \nabla J(w)^T(y - w) + \int_0^1 \int_0^t (y - w)^T \nabla^2 J(w + s(y - w))(y - w) ds dt.$$

Dalla convessità forte $\nabla^2 J \succeq \lambda I$, l'integrale doppio è maggiorato dal basso da:

$$\lambda \|y - w\|^2 \int_0^1 \int_0^t 1 ds dt = \frac{\lambda}{2} \|y - w\|^2.$$

Pertanto:

$$J(y) \geq J(w) + \nabla J(w)^T(y - w) + \frac{\lambda}{2} \|y - w\|^2. \quad (\text{A.6})$$

Minimizzando il membro destro rispetto a y si ottiene:

$$\min_y \left[J(w) + \nabla J(w)^T(y - w) + \frac{\lambda}{2} \|y - w\|^2 \right] = J(w) - \frac{1}{2\lambda} \|\nabla J(w)\|^2.$$

Dalla (A.6) segue che $J(y)$ è sempre maggiore o uguale al termine quadratico in y ; quindi il minimo di J sarà maggiore o uguale al minimo di tale termine. Poiché $y = w_*$ è il minimo di J e $J(w_*) = 0$, si ha:

$$0 = J(w_*) \geq J(w) - \frac{1}{2\lambda} \|\nabla J(w)\|^2,$$

da cui la tesi $\|\nabla J(w)\|^2 \geq 2\lambda J(w)$.

Si consideri ora la derivazione della disuguaglianza $J(w) \geq \frac{\lambda}{2L^2} \|\nabla J(w)\|^2$. Dalla rappresentazione integrale del gradiente che si ottiene valutando la (A.2) in $t = 1$:

$$\nabla J(w) = \int_0^1 \nabla^2 J(w_* + t(w - w_*))(w - w_*) dt. \quad (\text{A.7})$$

Passando alla norma:

$$\|\nabla J(w)\| = \left\| \int_0^1 \nabla^2 J(w_* + t(w - w_*))(w - w_*) dt \right\|.$$

Per la disuguaglianza triangolare in forma integrale:

$$\|\nabla J(w)\| \leq \int_0^1 \|\nabla^2 J(w_* + t(w - w_*))(w - w_*)\| dt.$$

Usando la limitatezza $\|\nabla^2 J\| \leq L$, si ha:

$$\|\nabla J(w)\| \leq \int_0^1 L \|w - w_*\| dt = L \|w - w_*\|. \quad (\text{A.8})$$

Da cui:

$$\|\nabla J(w)\| \leq L \|w - w_*\| \implies \|w - w_*\| \geq \frac{1}{L} \|\nabla J(w)\|. \quad (\text{A.9})$$

Elevando al quadrato (A.9):

$$\|w - w_*\|^2 \geq \frac{1}{L^2} \|\nabla J(w)\|^2. \quad (\text{A.10})$$

Dalla (A.3) invece:

$$J(w) = \int_0^1 (1-t) (w - w_*)^T \nabla^2 J(w_* + t(w - w_*)) (w - w_*) dt.$$

Usando la convessità forte $\nabla^2 J \succeq \lambda I$:

$$J(w) \geq \|w - w_*\|^2 \lambda \int_0^1 (1-t) dt.$$

L'integrale vale:

$$\int_0^1 (1-t) dt = \left[t - \frac{t^2}{2} \right]_0^1 = \frac{1}{2}.$$

Quindi:

$$J(w) \geq \frac{\lambda}{2} \|w - w_*\|^2. \quad (\text{A.11})$$

Combinando (A.10) e (A.11):

$$J(w) \geq \frac{\lambda}{2} \|w - w_*\|^2 \geq \frac{\lambda}{2} \cdot \frac{1}{L^2} \|\nabla J(w)\|^2 = \frac{\lambda}{2L^2} \|\nabla J(w)\|^2.$$

Questa è la seconda disuguaglianza cercata.

Con passaggi simili a quelli che sono stati fatti per dimostrare la (A.6) possiamo verificare che applicando il teorema di Taylor con resto integrale e usando $\nabla^2 J \preceq LI$ otteniamo

$$J(y) \leq J(w) + \nabla J(w)^T (y - w) + \frac{L}{2} \|y - w\|^2. \quad (\text{A.12})$$

4.2 Dimostrazione del fattore di correzione per popolazione finita

Il fattore $\frac{N-n_k}{N-1}$ è il **Fattore di Correzione per Popolazione Finita**. Per dimostrare formalmente il motivo per cui moltiplichiamo per questo fattore nel caso in cui accettassimo la possibilità di prendere più volte gli stessi dati nei batch possiamo prima dimostrare la necessità di questo fattore nel caso generale e poi applicarlo al nostro caso specifico: Consideriamo $\bar{X} = \frac{1}{n} \sum_{i \in \mathcal{S}} X_i = \frac{1}{n} \sum_{i=1}^N I_i X_i$ con

$$I_i = \begin{cases} 1 & \text{se } X_i \in \mathcal{S} \\ 0 & \text{se } X_i \notin \mathcal{S} \end{cases} \implies \begin{cases} \mathbb{E}[I_i] = \frac{n}{N}, & \mathbb{E}[I_i I_\ell] = \frac{n(n-1)}{N(N-1)} \\ \text{Var}(I_i) = \frac{n(N-n)}{N^2}, & \text{Cov}(I_i, I_\ell) = -\frac{n(N-n)}{N^2(N-1)} \end{cases}$$

$$\text{Var} \left(\sum_{i \in \mathcal{S}} X_i \right) = \sum_{i=1}^N X_i^2 \text{Var}(I_i) + \sum_{\substack{i, \ell \in \{1, \dots, N\} \\ i \neq \ell}} X_i X_\ell \text{Cov}(I_i, I_\ell)$$

Per verificarlo, basta espandere la definizione di varianza applicata alla combinazione lineare $\sum_{i=1}^N X_i I_i$: si scrive

$$\text{Var} \left(\sum_{i=1}^N X_i I_i \right) = \mathbb{E} \left[\left(\sum_{i=1}^N X_i (I_i - \mathbb{E}[I_i]) \right)^2 \right],$$

si sviluppa il quadrato della somma usando $\left(\sum_{i=1}^N a_i \right)^2 = \sum_{i=1}^N a_i^2 + \sum_{i \neq \ell} a_i a_\ell$ con $a_i = X_i (I_i - \mathbb{E}[I_i])$ (e questa servirà anche dopo, vera perché: $\underbrace{\sum_i \sum_\ell a_i a_\ell}_{(\sum_i a_i)^2} =$

$\sum_i a_i^2 + \sum_{i \neq \ell} a_i a_\ell \iff \sum_{i \neq \ell} a_i a_\ell = (\sum_i a_i)^2 - \sum_i a_i^2$), si applica la linearità dell'aspettazione e si riconoscono le formule per la varianza e covarianza di I

$$\begin{aligned} &= \frac{n(N-n)}{N^2} \sum_{i=1}^N X_i^2 - \frac{n(N-n)}{N^2(N-1)} \sum_{i \neq \ell} X_i X_\ell \\ &= \frac{n(N-n)}{N^2} \left[\sum_{i=1}^N X_i^2 - \frac{1}{N-1} \sum_{i \neq \ell} X_i X_\ell \right] \\ &\quad \sum_{i=1}^N X_i = N\mu \\ &\quad \sum_{i=1}^N X_i^2 = N(\mu^2 + \sigma^2) \end{aligned}$$

Infatti $\sigma^2 = \frac{1}{N} \sum_{i=1}^N (X_i - \mu)^2 \iff N\sigma^2 = \sum_{i=1}^N (X_i^2 - 2\mu X_i + \mu^2) \iff N\sigma^2 = \sum_{i=1}^N X_i^2 - 2\mu \sum_{i=1}^N X_i + N\mu^2 \iff N\sigma^2 = \sum_{i=1}^N X_i^2 - 2\mu(N\mu) + N\mu^2 \iff N\sigma^2 = \sum_{i=1}^N X_i^2 - 2N\mu^2 + N\mu^2 \iff N\sigma^2 = \sum_{i=1}^N X_i^2 - N\mu^2 \iff \sum_{i=1}^N X_i^2 = N\sigma^2 + N\mu^2 = N(\mu^2 + \sigma^2)$

$$\sum_{i \neq \ell} X_i X_\ell = \left(\sum_{i=1}^N X_i \right)^2 - \sum_{i=1}^N X_i^2 = N^2 \mu^2 - N(\mu^2 + \sigma^2)$$

$$\begin{aligned}
\sum_{i=1}^N X_i^2 - \frac{1}{N-1} \sum_{i \neq \ell} X_i X_\ell &= N(\mu^2 + \sigma^2) - \frac{1}{N-1} [N^2 \mu^2 - N(\mu^2 + \sigma^2)] \\
&= N(\mu^2 + \sigma^2) - \frac{N^2 \mu^2}{N-1} + \frac{N(\mu^2 + \sigma^2)}{N-1} \\
&= N(\mu^2 + \sigma^2) \left(1 + \frac{1}{N-1}\right) - \frac{N^2 \mu^2}{N-1} = N(\mu^2 + \sigma^2) \frac{N}{N-1} - \frac{N^2 \mu^2}{N-1} \\
&= \frac{N^2(\mu^2 + \sigma^2)}{N-1} - \frac{N^2 \mu^2}{N-1} = \frac{N^2 \sigma^2}{N-1} = N \sigma^2 \frac{N}{N-1} \\
\text{Var} \left(\sum_{i \in \mathcal{S}} X_i \right) &= \frac{n(N-n)}{N^2} \cdot N \sigma^2 \frac{N}{N-1} = \frac{n(N-n)}{N-1} \sigma^2 \\
\text{Var}(\bar{X}) &= \frac{1}{n^2} \text{Var} \left(\sum_{i \in \mathcal{S}} X_i \right) = \frac{1}{n^2} \cdot \frac{n(N-n)}{N-1} \sigma^2 = \frac{\sigma^2}{n} \cdot \frac{N-n}{N-1}
\end{aligned}$$

Nella dimostrazione generale con le variabili X_i , poniamo:

- $X_i \longleftrightarrow \nabla \ell(w_k; i)$ (gradiente della funzione di perdita sull'esempio i -esimo, valutato in w_k)
- $\mu \longleftrightarrow \nabla J(w_k) = \mathbb{E}_{i \sim \text{Unif}\{1, \dots, N\}} [\nabla \ell(w_k; i)] = \frac{1}{N} \sum_{i=1}^N \nabla \ell(w_k; i)$
- $\sigma^2 \longleftrightarrow \mathcal{V}_j(w_k)$ per ogni $j = 1, \dots, m$, dove

$$\mathcal{V}(w_k) := \frac{1}{N} \sum_{i=1}^N (\nabla \ell(w_k; i) - \nabla J(w_k))^2 \in \mathbb{R}^m$$

è la versione vettoriale di σ^2 nel caso multidimensionale.

Applicando la formula generale $\text{Var}(\bar{X}) = \frac{\mathcal{V}}{n_k} \cdot \frac{N-n_k}{N-1}$ a ogni componente $j = 1, \dots, m$:

$$\text{Var}(\nabla J_{\mathcal{S}_k}(w_k)) = \frac{\mathcal{V}(w_k)}{n_k} \cdot \frac{N-n_k}{N-1}$$

e quindi, in norma $\|\cdot\|_1$:

$$\|\text{Var}(\nabla J_{\mathcal{S}_k}(w_k))\|_1 = \frac{\|\mathcal{V}(w_k)\|_1}{n_k} \cdot \frac{N-n_k}{N-1}.$$

Poiché $\mathcal{V}(w_k)$ è ignoto, lo stimiamo con $\hat{\mathcal{V}}(w_k) := \frac{1}{n_k-1} \sum_{i \in \mathcal{S}_k} (\nabla \ell(w_k; i) - \nabla J_{\mathcal{S}_k}(w_k))^2$ ovvero la varianza campionaria, ottenendo la condizione di controllo della varianza:

$$\frac{\|\hat{\mathcal{V}}\|_1}{n_k} \cdot \frac{N-n_k}{N-1} \leq \theta^2 \|\nabla J_{\mathcal{S}_k}(w_k)\|^2.$$

4.3 Dimostrazione delle formule per α_k e β_k nel Metodo del Gradiente Coniugato

Consideriamo il funzionale quadratico

$$\phi(x) = \frac{1}{2}x^T A x - b^T x, \quad A = A^T > 0. \quad (\text{A.13})$$

Sia x^* il minimo, soluzione di $Ax^* = b$. Definiamo

$$\begin{cases} e^k = x^* - x^k, \\ r^k = b - Ax^k, \end{cases} \quad \begin{matrix} Ax^* = b \\ \implies \end{matrix} \quad r^k = Ae^k. \quad (\text{A.14})$$

L'aggiornamento del metodo è

$$\begin{cases} x^{k+1} = x^k + \alpha_k p^k, \\ p^k = r^k + \beta_k p^{k-1}, \quad p^0 = r^0 \end{cases} \quad (\text{A.15})$$

da cui, per la definizione di e^k ,

$$e^{k+1} = e^k - \alpha_k p^k. \quad (\text{A.16})$$

Il metodo minimizza $\|e^{k+1}\|_A^2$ nel piano $e^k + \text{span}(p^k, p^{k-1})$. Definiamo

$$F(\alpha, \gamma) = \|e^k + \alpha p^k + \gamma p^{k-1}\|_A^2. \quad (\text{A.17})$$

Poiché $\|z\|_A^2 = z^T A z$, con $z = e^k + \alpha p^k + \gamma p^{k-1}$:

$$\begin{aligned} F(\alpha, \gamma) &= (e^k + \alpha p^k + \gamma p^{k-1})^T A (e^k + \alpha p^k + \gamma p^{k-1}) \\ &= \|e^k\|_A^2 + 2\alpha(e^k)^T A p^k + 2\gamma(e^k)^T A p^{k-1} \\ &\quad + \alpha^2 \|p^k\|_A^2 + 2\alpha\gamma(p^k)^T A p^{k-1} + \gamma^2 \|p^{k-1}\|_A^2. \end{aligned} \quad (\text{A.18})$$

Annullando le derivate parziali:

$$\begin{cases} (e^k)^T A p^k + \alpha \|p^k\|_A^2 + \gamma(p^k)^T A p^{k-1} = 0, \\ (e^k)^T A p^{k-1} + \alpha(p^k)^T A p^{k-1} + \gamma \|p^{k-1}\|_A^2 = 0. \end{cases} \quad (\text{A.19})$$

Per la scelta di α_{k-1} (minimo lungo p^{k-1}):

$$\frac{d}{d\alpha} \phi(x^{k-1} + \alpha p^{k-1}) \Big|_{\alpha=\alpha_{k-1}} = 0 \implies \nabla \phi(x^k)^T p^{k-1} = 0 \implies (r^k)^T p^{k-1} = 0. \quad (\text{A.20})$$

Usando $r^k = Ae^k$ e la simmetria di A :

$$(e^k)^T A p^{k-1} = (Ae^k)^T p^{k-1} = (r^k)^T p^{k-1} = 0. \quad (\text{A.21})$$

Si noti l'ordine logico della deduzione: si applica prima la (A.21) alla seconda equazione del sistema (A.19) il termine $(e^k)^T A p^{k-1}$ si annulla e si ottiene $\gamma \|p^{k-1}\|_A^2 = 0$, da cui $\gamma = 0$; la proprietà (A.23), cioè $(p^k)^T A p^{k-1} = 0$, segue poi come conseguenza della prima equazione.

La minimizzazione nel piano implica le condizioni di ortogonalità

$$e^{k+1} \perp_A p^k \quad \text{e} \quad e^{k+1} \perp_A p^{k-1}. \quad (\text{A.22})$$

Dalla seconda, con $e^{k+1} = e^k - \alpha_k p^k$ e (A.21):

$$(e^k - \alpha_k p^k)^T A p^{k-1} = 0 \implies -\alpha_k (p^k)^T A p^{k-1} = 0 \xrightarrow{\alpha_k \neq 0} (p^k)^T A p^{k-1} = 0. \quad (\text{A.23})$$

Sfruttando (A.21) e (A.23) nel sistema (A.19):

$$\begin{cases} (e^k)^T A p^k + \alpha \|p^k\|_A^2 = 0, \\ \gamma \|p^{k-1}\|_A^2 = 0, \end{cases} \implies \gamma = 0, \quad \alpha = -\frac{(e^k)^T A p^k}{\|p^k\|_A^2}. \quad (\text{A.24})$$

Ora, $(e^k)^T A p^k = (r^k)^T p^k$ e da (A.20) $r^k \perp p^{k-1}$; quindi

$$(r^k)^T p^k = (r^k)^T (r^k + \beta_k p^{k-1}) = \|r^k\|^2. \quad (\text{A.25})$$

Pertanto

$$\alpha_k = \frac{\|r^k\|^2}{\|p^k\|_A^2} = \frac{(r^k)^T r^k}{(p^k)^T A p^k}. \quad (\text{A.26})$$

Per β_k , da (A.23) e $p^k = r^k + \beta_k p^{k-1}$:

$$(r^k + \beta_k p^{k-1})^T A p^{k-1} = 0 \implies \beta_k = -\frac{(r^k)^T A p^{k-1}}{(p^{k-1})^T A p^{k-1}}. \quad (\text{A.27})$$

Dall'aggiornamento del residuo:

$$r^k = r^{k-1} - \alpha_{k-1} A p^{k-1} \implies A p^{k-1} = \frac{r^{k-1} - r^k}{\alpha_{k-1}}. \quad (\text{A.28})$$

Al numeratore di (A.27):

$$(r^k)^T A p^{k-1} = \frac{(r^k)^T r^{k-1} - (r^k)^T r^k}{\alpha_{k-1}} \stackrel{(r^k)^T r^{k-1}=0}{=} -\frac{(r^k)^T r^k}{\alpha_{k-1}}. \quad (\text{A.29})$$

Dimostrazione di $(r^k)^T r^{k-1} = 0$: Mostriamo per induzione che $r^k \perp p^j$ per $j \leq k-1$.

- Base: $k=1$, (A.20) dà $r^1 \perp p^0$.
- Passo: supponiamo $r^{k-1} \perp p^j$ per $j \leq k-2$. Per $r^k = r^{k-1} - \alpha_{k-1} A p^{k-1}$ e $j \leq k-2$:

$$(r^k)^T p^j = (r^{k-1})^T p^j - \alpha_{k-1} (p^{k-1})^T A p^j = 0.$$

Inoltre (A.20) dà $(r^k)^T p^{k-1} = 0$. Quindi $r^k \perp p^j$ per $j \leq k-1$.

In particolare $r^k \perp p^{k-2}$. Da $p^{k-1} = r^{k-1} + \beta_{k-1} p^{k-2}$:

$$0 = (r^k)^T p^{k-1} = (r^k)^T r^{k-1} + \beta_{k-1} (r^k)^T p^{k-2} \implies (r^k)^T r^{k-1} = 0. \quad (\text{A.30})$$

Al denominatore di (A.27), usando (A.26) per $k-1$:

$$(p^{k-1})^T A p^{k-1} = \frac{(r^{k-1})^T r^{k-1}}{\alpha_{k-1}}. \quad (\text{A.31})$$

Sostituendo (A.29) e (A.31) in (A.27):

$$\beta_k = -\frac{-\frac{(r^k)^T r^k}{\alpha_{k-1}}}{\frac{(r^{k-1})^T r^{k-1}}{\alpha_{k-1}}} = \frac{(r^k)^T r^k}{(r^{k-1})^T r^{k-1}}. \quad (\text{A.32})$$

Abbiamo quindi dimostrato che le condizioni di ortogonalità (A.22) portano a:

$$\alpha_k = \frac{(r^k)^T r^k}{(p^k)^T A p^k}, \quad \beta_k = \frac{(r^k)^T r^k}{(r^{k-1})^T r^{k-1}} \quad (\text{A.33})$$

con

$$p^k = r^k + \beta_k p^{k-1}, \quad x^{k+1} = x^k + \alpha_k p^k. \quad (\text{A.34})$$

Equivalenza delle formule per α_k . Nel documento (Nota Tecnica) si afferma che le formule $\alpha_k = \frac{r_k^T p_k}{p_k^T A p_k}$ e $\alpha_k = \frac{r_k^T r_k}{p_k^T A p_k}$ sono equivalenti. Dimostriamo questa equivalenza.

Prendendo la condizione di stazionarietà (A.20), ovvero $(r^k)^T p^{k-1} = 0$, e sostituendo l'aggiornamento del residuo (A.28), $r^k = r^{k-1} - \alpha_{k-1} A p^{k-1}$, si ottiene:

$$\begin{aligned} 0 &= (r^k)^T p^{k-1} = (r^{k-1} - \alpha_{k-1} A p^{k-1})^T p^{k-1} = (r^{k-1})^T p^{k-1} - \alpha_{k-1} (p^{k-1})^T A p^{k-1} \\ &\implies (r^{k-1})^T p^{k-1} - \alpha_{k-1} (p^{k-1})^T A p^{k-1} = 0. \end{aligned} \quad (\text{A.35})$$

Ora, l'aggiornamento del residuo nel metodo del gradiente coniugato è:

$$r_k = r_{k-1} - \alpha_{k-1} A p_{k-1}. \quad (\text{A.36})$$

Moltiplichiamo scalarmente la (A.36) per p_{k-1} :

$$r_k^T p_{k-1} = r_{k-1}^T p_{k-1} - \alpha_{k-1} p_{k-1}^T A p_{k-1}.$$

Ma il membro destro è esattamente zero per la (A.35). Quindi:

$$r_k^T p_{k-1} = 0. \quad (\text{A.37})$$

Questo risultato è valido per ogni $k \geq 1$ e deriva unicamente dalla line search esatta al passo precedente.

Ora, nella direzione di ricerca del gradiente coniugato si ha:

$$p_k = r_k + \beta_k p_{k-1}. \quad (\text{A.38})$$

Moltiplichiamo scalarmente la (A.38) per r_k :

$$r_k^T p_k = r_k^T r_k + \beta_k r_k^T p_{k-1}.$$

Usando la (A.37), il secondo termine si annulla:

$$r_k^T p_k = r_k^T r_k. \quad (\text{A.39})$$

Sostituendo la (A.39) nella formula $\alpha_k = \frac{r_k^T p_k}{p_k^T A p_k}$, otteniamo:

$$\alpha_k = \frac{r_k^T r_k}{p_k^T A p_k}. \quad (\text{A.40})$$

Abbiamo quindi dimostrato che:

$$\frac{r_k^T p_k}{p_k^T A p_k} = \frac{r_k^T r_k}{p_k^T A p_k}.$$

Le due formule per α_k sono quindi equivalenti. La formula con $r_k^T r_k$ è preferita nelle implementazioni numeriche perché è numericamente più stabile, poiché non dipende dall'ortogonalità numerica di $r_k^T p_{k-1}$, che in presenza di errori di arrotondamento non è esattamente zero.

4.4 Spiegazione delle condizioni sul passo α_k

La condizione di Armijo introduce un limite superiore per il valore della funzione lungo la direzione di ricerca: il passo α è accettato se $f(x_k + \alpha p)$ si trova al di sotto della retta $f(x_k) + c_1 \alpha \nabla f(x_k)^T p$, e rifiutato altrimenti. La condizione di curvatura di Wolfe impone invece un limite inferiore per la derivata $f'(\alpha)$ lungo la direzione.

La condizione di Armijo richiede che

$$f(x_k + \alpha p) \leq f(x_k) + c_1 \alpha \nabla f(x_k)^T p,$$

con $c_1 \in (0, 1)$ (tipicamente 10^{-4}). La retta al membro destro ha pendenza $c_1 \nabla f(x_k)^T p$, meno ripida della tangente in $\alpha = 0$, e funge da *upper bound*: passi che fanno superare questa retta vengono rifiutati e ridotti. Più c_1 è piccolo, più la condizione è debole e più passi grandi sono accettati.

La condizione di curvatura di Wolfe, con $g(\alpha) = f(x_k + \alpha p)$ e $g'(\alpha) = \nabla f(x_k + \alpha p)^T p$, richiede

$$g'(\alpha) \geq c_2 g'(0), \quad c_2 \in (c_1, 1),$$

dove $g'(0) = \nabla f(x_k)^T p < 0$. Poiché $c_2 < 1$, $c_2 g'(0)$ è meno negativo di $g'(0)$: la condizione è un *lower bound* sulla derivata, che scarta i passi troppo piccoli (per $\alpha \rightarrow 0^+$, $g'(\alpha) \rightarrow g'(0)$, che non soddisfa la disuguaglianza).

La condizione $0 < c_1 < c_2 < 1$ garantisce che le due disuguaglianze siano compatibili: Armijo (limite superiore su f) e Wolfe (limite inferiore su g') definiscono un intervallo non vuoto di α accettabili. Se $c_1 \geq c_2$, le due condizioni possono diventare contraddittorie. In sintesi, Armijo impedisce passi troppo grandi, Wolfe impedisce passi troppo piccoli.

Riferimenti bibliografici

- [1] R. H. Byrd, G. M. Chin, J. Nocedal, and Y. Wu, “Sample size selection in optimization methods for machine learning,” *Mathematical Programming, Series A*, vol. 134, no. 1, pp. 127–155, 2012.
- [2] L. Bottou and O. Bousquet, “The tradeoffs of large scale learning,” in *Advances in Neural Information Processing Systems (NIPS)*, vol. 20, 2008.